DLSG 2026-10-13

c.f. "Harness Learning Enables Generalizable Test-Time Adaptation" arXiv:2609.35738 (Sept. 28, 2026)

Context Language Models

Rulin Shao[1,2], **Shannon Zejiang Shen**[3], **Junjie Oscar Yin**[1,2], **Yuetai Li**[1], **Minheng Wang**[1], **Hamish Ivison**[1], **Radha Poovendran**[1], **Nathan Lambert**[4], **Teng Xiao**[1], **Mike Lewis**[2], **Wen-tau Yih**[2], **Luke Zettlemoyer**[1,2], **Pang Wei Koh**[1]

[1]University of Washington, [2]Meta Superintelligence Labs, [3]MIT, [4]Trillium Labs

We introduce **Context Language Models (CLMs)**, language models that natively manage their own context. We implement this by treating the **context as a file** and allowing the model to make unrestricted updates to this file. This allows the model to learn what is most important to maintain in context, and naturally extends to multi-agent systems where multiple agent contexts coexist as files. Building CLMs **zero-shot** with existing models outperforms SOTA context-management strategies across a variety of tasks: 11.4% higher accuracy with 21.5% fewer FLOPs on BrowseComp-Plus, 5% higher scores with 59% fewer FLOPs on 12-hour EdgeBench, and 65% greater improvement with the same compute on a 24-hour multi-repository agent-swarm task. Moreover, by shifting context management from external harness control to intrinsic model behavior, CLMs naturally enable both **in-context and parametric learning** of context-management strategies. We show that CLMs can be steered with natural-language instructions evolved through a standard skill-optimization loop, improving held-out accuracy by up to 35.9 points on a context-management task while reducing compute. We also introduce an online reinforcement learning method for CLMs, improving Qwen3.5-9B performance on BrowseComp-Plus by 47.6% while using 12% fewer FLOPs. Finally, we co-design **Suffix Cache Reuse** for CLM serving, further reducing server-side compute by 35% relative to standard SGLang at matched performance.

Date: September 30, 2026
Correspondence: Rulin Shao at rulin@cs.washington.edu
Code: https://github.com/facebookresearch/context-language-models

Meta

arXiv:2609.37725v1 [cs.AI] 29 Sep 2026

Prompt: https://github.com/facebookresearch/context-language-models/blob/main/clm/clm_harness/clm_agent/prompts.yaml

1 Introduction

Despite the fact that context is the cornerstone that allows a language model (LM) to process and retain information over time, context management is not traditionally a native LM capability. Instead, prior work mostly relies on harnesses, either hand-engineered (Cassano and Rush, 2026; OpenAI, 2026a; Merrill et al., 2026) or optimized offline by agents (Lee et al., 2026). Recent work adds a constrained set of tools with fixed strategies such as compaction, offloading, and retrieval, to the agent's action space (Yan et al., 2026; Yu et al., 2026; Li et al., 2026c; Liu et al., 2026; Zhang et al., 2025a). In contrast, we show that giving LMs unrestricted access to manage their own context outperforms human-designed baselines, enabling adaptive and creative context-management strategies to emerge. Our findings echo The Bitter Lesson (Sutton, 2019): we should let LMs search for and learn better strategies that go far beyond existing human priors.

Concretely, we introduce **Context Language Models (CLMs)**, which are natively capable of managing their own context. We show existing LMs can be turned into strong CLMs and can be further improved through in-context learning and reinforcement learning. Formally, a CLM parametrized by θ makes context an artifact of the LM: $c_{t+1} = f_\theta^{\text{CLM}}(c_t)$, where c_t is the context at turn t and f_θ^{CLM} can be an arbitrary function controlled by CLMs. In contrast, a standard LM simply appends new tokens to the existing context: $c_{t+1} = c_t \oplus f_\theta^{\text{LM}}(c_t)$.

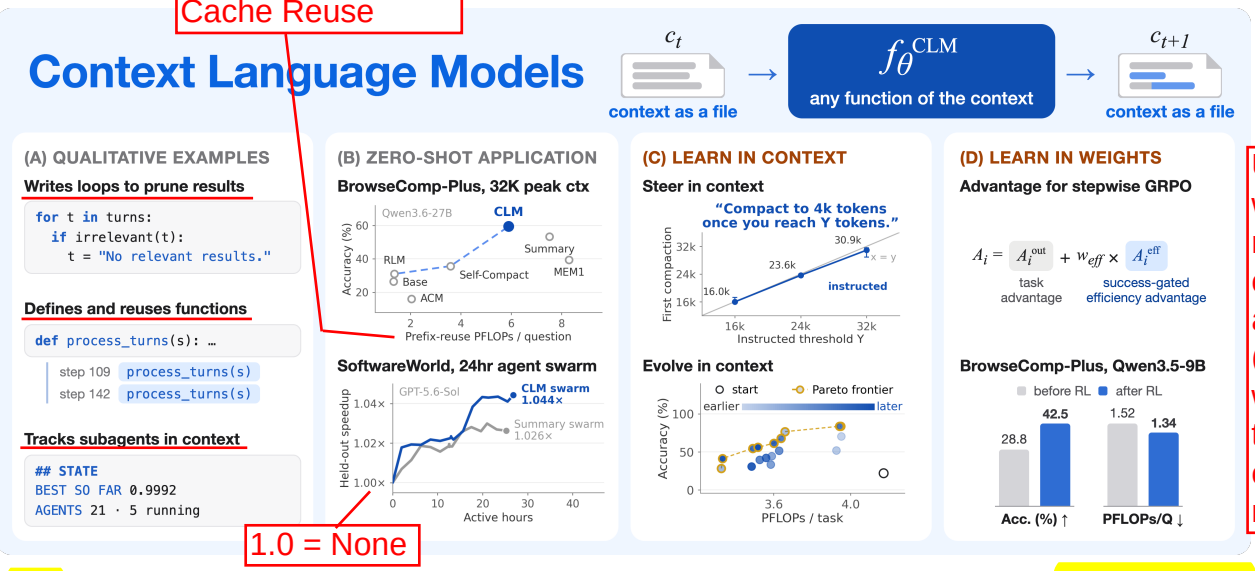
We implement CLMs by treating **context as a file**. Specifically, we mirror the context into a storage space with LM write access. The LM can either append newly generated tokens or use Bash to freely edit the context file, with each modification immediately synchronized to the LM's live context for the next turn. This design naturally extends to multi-agent systems, where multiple context files can coexist and be managed by CLMs for **agent-swarm** or **subagent** workloads.

Poor wording, but I think "next turn" means context can be edited after each tool call. This is typically more frequent than at each user prompt, but much less frequent than after each token generation. Figure 3 seems to show context edits at agent steps 40, 109, 455 & 1509 within autonomous agent execution. See also Section 4.1 (which is also ambiguous).

(C) is response to context mgmt text in context vs. (D) is context mgmt. coded in weights via RL
(C) top - when it compacts vs. instruction (perfect). (C) bottom - context instructions vs. FLOPS - Pareto frontier
(D) Equation and results - task adv, is 0,1 for task success. 2nd term is computational efficiency. Bottom - better, faster results.

prompt: only shows a regex example - does not mention: writing loops, writing functions (invoked 37 times), scoreboards (updates 163x)

without Suffix Cache Reuse



1.0 = None

Unstated whether (D) results could be achieved by (C) (i.e. without RL training costs) - so maybe.

Figure 1 Context Language Models (CLMs) natively manage their own context by treating context as a file. CLMs work out of the box and can be further improved through in-context learning and reinforcement learning. (a) Qualitative examples of creative context-management behaviors introduced by CLM. (b) Out of the box, CLMs improve performance at lower cost on BrowseComp-Plus, a deep-research benchmark, and Software World, where an agent swarm jointly optimizes six interdependent repositories. (c) CLMs can follow textual instructions to adopt corresponding context-management strategies (top) or evolve better strategies through a skill-evolution loop on ContextBench (bottom). (d) CLMs can explore and internalize context-management strategies through online reinforcement learning. By using a success-gated efficiency advantage for stepwise GRPO, we improve both accuracy and efficiency for CLMs simultaneously.

Arbitrary context edits in CLMs pose new challenges for existing serving systems, which typically only reuse cached states for matching prefixes, forcing re-prefilling after in-the-middle edits. We account for this by introducing **prefix-reuse FLOPs**, which capture the trajectory-wide inference costs of decoding, prefilling, and re-prefilling in standard LM serving, and show that CLMs remain more compute-efficient under standard serving through better context management. We further develop **Suffix Cache Reuse (SCR)**, which reuses cached states beyond the matching prefix to reduce re-prefilling while empirically preserving task performance. We also introduce **ContextBench** as a diagnostic benchmark that decouples context management from reasoning and knowledge, revealing the limitations of existing context management methods.

We show that CLMs, applied **zero-shot** to models like Qwen3.6-27B and GPT5.6-Sol, outperform existing baselines and task-specific harnesses across diverse long-horizon tasks, ranging from hundreds to thousands of turns and up to 24 hours of runtime. Compared with existing harness-defined and action-based methods, CLM achieves 11.4% higher accuracy with 21.5% fewer prefix-reuse FLOPs than the strongest baseline on the deep-research benchmark BrowseComp-Plus (Chen et al., 2025), while matching the strongest baseline's accuracy on the terminal-coding benchmark TerminalBench 2.1 (Merrill et al., 2026) with 29.5% fewer FLOPs. On mathematical optimization tasks, CLM outperforms specialized evolutionary harnesses such as OpenEvolve (Sharma, 2025) by up to 16.8% (Heilbronn) and 3.0% (circle packing). On long-running software optimization, CLM outperforms Codex-style summarization: on 12-hour EdgeBench (Zhu et al., 2026) (a 10-task subset), CLM scores 5% higher while using 59% fewer prefix-reuse FLOPs, and on a 24-hour six-repository agent-swarm task, it achieves 65% greater end-to-end speedup at the same compute. Moreover, when Suffix Cache Reuse is further applied, it helps reduce server-side compute by 35% with matched performance compared with standard SGLang serving. Qualitatively, we find that CLMs come up with novel emergent behaviors such as defining and maintaining trackers for multi-agent orchestration, introducing a new chat role for internal notes, and defining reusable context-management functions.

By shifting context management from external harness control to intrinsic model behavior, CLMs naturally enable **in-context learning** and **parametric learning** for context management. We first show that users can **steer** context management simply by telling the agent their desired strategy. In addition, CLMs can **evolve** an in-context skill document that captures useful context-management procedures for future reuse, improving

DeepSeekMath

held-out accuracy on ContextBench by up to 35.9 points at lower compute. For training, we introduce a **success-gated efficiency advantage** in stepwise GRPO (Shao et al., 2024) that rewards efficient CLM trajectories among successful ones. Training Qwen3.5-9B on deep research tasks in this manner improves CLM from 28.8% to 42.5% on BrowseComp-Plus, outperforming a Codex-style summary harness trained with the same recipe by 0.4 points while using 38.8% fewer FLOPs. Overall, we show that by treating context management as a native LM capability, CLMs enable more effective and efficient strategies to be searched for and learned.

2 Related Work

From harness-defined to action-based context management. Most existing harnesses compact accumulated histories according to a fixed harness policy (Cassano and Rush, 2026; OpenAI, 2026a; Merrill et al., 2026; Zhou et al., 2026), such as at a predefined length threshold or at every turn. Recent work gives the model increasing control through human-defined actions: AutoCompact (Zhang et al., 2026b) and Self-Compact (Li et al., 2026b) let the model decide when to compact; Context-as-a-Tool (Liu et al., 2026) exposes model-triggered compaction over a predefined portion of the context; ACM (Li et al., 2026c) adds model-triggered offloading and retrieval; and Sculptor (Li et al., 2026a) lets the model select context fragments to operate on. Across this progression, model autonomy increases but remains restricted to a human-defined action space. Our work pushes this autonomy to its limit by granting the model full agency over its context.

Context as a REPL variable. Recursive Language Models (RLMs) (Zhang et al., 2025a) treat a long input as a read-eval-print loop (REPL) variable that LMs can recursively access on demand. This addresses when and what information to *read* into the context. However, RLMs do not address how the live context itself should be managed. Retrieved information is still appended to the live context, which continues to grow over time. In contrast, our work makes the live context editable, giving CLMs full control over their context.

We provide extended related work in Appendix A, with more detailed comparisons to existing context-management baselines and a discussion of meta-harness optimization, reinforcement learning, cache reuse, and the relationship between context management and external memory.

3 Pilot Study with ContextBench: A Diagnostic for Context Management

We start with a pilot study showing how existing context management strategies can fail in simple tasks. To isolate context management from other reasoning or knowledge capabilities, we developed **ContextBench**, a diagnostic evaluation suite with the four synthetic tasks shown in Figure 2: **Needle Retention** tests selective verbatim retention, simulating the need to preserve important information over time; **Sudoku Sketchpad** tests surgical in-place updates to the live context by maintaining a Sudoku board as users stream in moves; **KV Store** and **Log Triage** test exact recall through offloading and retrieval of massive values and working logs. We evaluate ContextBench with several context-management strategies, including Mini-SWE-Agent (Yang et al., 2024) (the base harness without context management), Codex-style Summary (OpenAI, 2026a), Context Folding (Sun et al., 2025), and RLM, Self-Compact, and ACM, as introduced in Section 2. We also evaluate CLM, which will be introduced in Section 4. Details of the evaluation and qualitative examples for ContextBench are provided in Appendix D.

We fix the context limit at 32K and vary the context pressure (the ratio of input volume to context limit) up to 24×. The results in Figure 2 show that these fixed strategies cannot adapt well to the live context: Summary-based compaction can lose or hallucinate information on Needle Retention and Sudoku Sketchpad; methods without flexible in-place editing must regenerate the full Sudoku state for every fine-grained user edit; and standard coding tools can offload information on KV Store and Log Triage but cannot evict it from the live context on demand. As a result, none of the existing methods performs perfectly even on these simple tasks. These failures motivate fully adaptive, model-controlled context management.

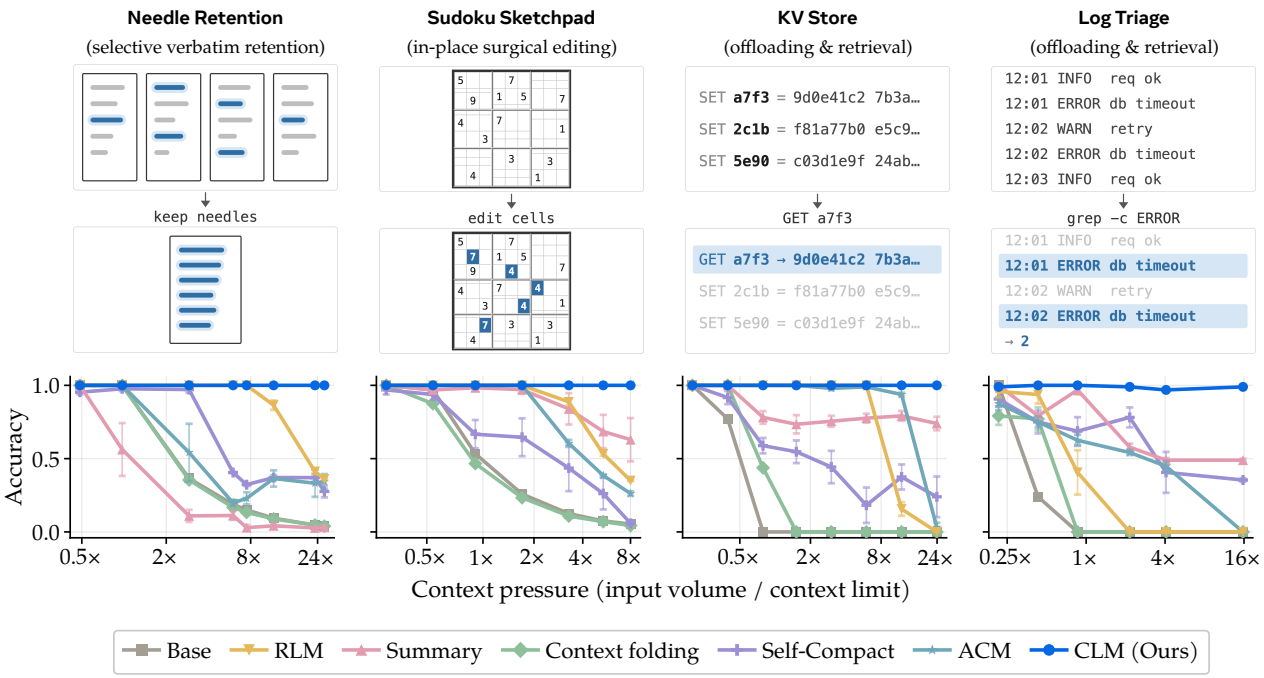


Figure 2 Illustration of the four tasks in ContextBench and a performance comparison of CLM against baselines using GPT-5.4 with a 32K context limit.

4 Context Language Models (CLMs)

4.1 Formal Definition and Implementation with Context as a File

Context Language Models (CLMs) generalize the append-only context transition of a standard LM to a model-controlled context transition. Standard LMs append model output to the current context:

$$c_{t+1} = c_t \oplus f_\theta^{\text{LM}}(c_t), \tag{1}$$

where $\oplus$ denotes concatenation. In contrast, CLMs delegate full responsibility for maintaining the context to the CLM itself, directly creating the next context:

$$c_{t+1} = f_\theta^{\text{CLM}}(c_t), \tag{2}$$

where f_θ^{CLM} can be an arbitrary function controlled by CLMs. Eq. 2 subsumes prior approaches that expose a set of context-management tools through the harness. However, prior work requires context-management functions to be predefined in the harness. Our work instead makes CLMs responsible for defining these functions themselves as a *meta-capability*, either implicitly through their planning or explicitly as reusable functions, with one explicit example shown in Figure 3d.

Context-as-a-file implementation for CLMs. To implement CLMs, we mirror the LM's live context as a directly editable file and provide its path in the system prompt. The LM can edit this file using general Bash commands, just as it would edit other files in storage. Unlike ordinary files, edits to the context file are automatically synchronized with the LM's context and sent to the LLM server for continued generation. When the LM does not edit the context file, the generated tokens are appended to the existing context by default. This implementation balances context reuse with the flexibility to edit the context.

Multi-agent extensions of CLMs. Our implementation naturally extends to multi-agent workflows by allowing multiple context files to coexist and remain synchronized with their respective LLM servers. For example, an **agent swarm** can be implemented by initializing the workspace with multiple context files, while **subagents** can be initialized and terminated by creating and deleting additional context files.

Prompt:
https://github.com/facebookresearch/context-language-models/blob/main/clm/clm_harness/clm_agent/prompts.yaml

No proven benefit in allowing multiple agents to maintain and coordinate separate contexts improves performance beyond what one agent could achieve. EdgeBench - multi-agent worse (MA). Math optimization & winner (circle pack: MA-CLM, Heilbronn 1-CLM, minmax/min-distance =, erdos overlap 1-CLM)

(a) CLM builds in-context scoreboards and trackers to orchestrate and monitor subagents with in-place editing.

```
open(p,"w").write("""[[CTX_TURN 1 role=assistant]]
## STATE — Erdős Minimum Overlap Problem (compact)
LEDGER TOP: 0.9992491 ...
AGENTS: 21 launched, 5 currently running ...
KEY FILES: /workspace/subctx_4/h_best_final.npy ...
FINDINGS: All methods plateau at 0.381157 ...""")
```

Erdős minimum overlap, step 455

```
new="""## ORCHESTRATOR STATE (compact)
Budget: 7/100 used. All 5 slots BUSY (subctx_0..4 ...
Dead ends: simple grid(0.822), hexagonal(0.9977) ...
Next: score my own candidates while workers run ..."""
```

Circle packing $N = 26$, steps 194, 198

(c) CLM writes for loops to remove past irrelevant search results or compact overly long outputs when creating a new view.

```
for t in turns[1:]: ...
    result += f'\n[[CTX_TURN search]]\nSearched: {m.group(1).strip
↪ ()}. No relevant results.\n'
```

BrowseComp-Plus, step 1509

```
while i < len(lines): ...
    if len(body) > 500: ...
        if 'bcp_search' in body: ...
            result.append(f"[Searched: {query}]")
        elif 'bcp_get_document' in body: ...
            result.append(f"[Retrieved doc {docid}]")
    else:
        result.extend(body_lines)
```

BrowseComp-Plus, step 13

(b) CLM creates a new role alongside the original template roles for its own internal notes.

```
re.sub(r"\[\[CTX_TURN 4 .*?(?=\[\[CTX_TURN 16)",
  """[[CTX_TURN 4 role=notes]]
STATUS: ... James Gallagher (docid=58939) ...""")
```

BrowseComp-Plus, step 40

(d) CLM defines and reuses a function to conveniently compact old results with a reference to its maintained note.

```
progress = """[Search progress: VERIFIED ... NEXT: ...]"""
s = re.sub(..., progress, s)

def compact_turns(text):
    return re.sub(..., lambda m: m.group(0).split('\n')[0]
        + '\n[search results - see progress note]', text)
s = compact_turns(s)
```

BrowseComp-Plus, step 109

(e) CLM reproduces effective behaviors from existing baselines, preserving important facts and future TODOs in the summary.

```
re.sub(r"\[\[CTX_TURN 2.*",
  "[SUMMARY: ... Kader Asmal Excellence Award
  launched 2011 by Mrs A Motshekga ...]")
```

BrowseComp-Plus, step 20

```
new="""[EXPLORATION LEDGER - 86 scored attempts ...
Best score: 0.9931 (sum_radii=2.6177) from ...
UNTRIED IDEAS (priority order):
1. Gradient clipping norm=1.0 with 12k steps
2. Try lam=2200+uniform(0,2800) with 12k steps ..."""
```

Circle packing $N = 26$, step 332

Figure 3 Qualitative examples of CLM context-management behaviors. CLMs treat context as a file and can arbitrarily edit it using general code interface.

Qualitative examples. We show qualitative examples of CLMs managing context as a file in Figure 3, revealing both novel context-management behaviors and effective compaction strategies. For multi-agent orchestration, CLM maintains an in-context scoreboard and updates agent status through 163 in-place edits while keeping the context at only 6–8K tokens (a). It can create new internal roles such as "notes" when rewriting its context (b), and use loops to remove irrelevant search results or compact overlong observations (c). CLM can also define and reuse helper functions: in (d), it invokes 'compact_turns' 37 times to maintain a progress note while compacting detailed observations. Finally, it reproduces effective compaction behaviors by compressing 21K tokens into answer-relevant summaries or preserving untried ideas for future explorations (e). We collected these examples from the zero-shot CLM evaluation experiments in Section 5.1.

Efficiency metrics for CLMs. A common serving optimization is prefix-cache reuse, in which cached states are reused for matching prefixes, while all tokens from the first prefix mismatch onward must be re-prefilled, as can occur after an in-the-middle edit. To account for this, we measure theoretical inference FLOPs using a metric we call **prefix-reuse FLOPs** (see Appendix C for details). Formally,

this is without Suffix Cache Reuse (per 4.3)

$$\text{FLOPs}_{\text{prefix-reuse}} = \underbrace{\text{FLOPs}_{\text{prefill}}(\text{unmatched context suffix})}_{\text{tokens from the first prefix mismatch onward}} + \underbrace{\text{FLOPs}_{\text{decode}}(\text{generated tokens})}_{\text{new output tokens}}. \quad (3)$$

4.2 In-Context Learning and Reinforcement Learning for CLMs

By treating context management as an LM-native capability, CLMs can learn better strategies in context or in weights.

Steering CLMs with in-context instruction or skill documents. Let s denote an in-context instruction or skill document. CLMs can be steered by simply providing s as additional in-context guidance to the CLM:

$$c_{t+1} = f_\theta^{\text{CLM}}(c_t; s). \quad (4)$$

Uses (2) but adds 's' to $f_0^{\text{CLM}}()$

Consider three different skills for s: (i) Conservative compression. Do not edit context until it exceeds 48,000 tokens. Preserve exact experimental results, error messages, and failed attempts whenever possible. (ii) Aggressive compression. When your context exceeds 16,000 tokens, summarize completed investigations and remove detailed tool outputs. Retain only the current plan and essential facts. (iii) Structured memory. Maintain a compact state table. Keep exact experimental results in external files, store their paths in the active context, and delete redundant tool outputs after recording their conclusions.

Algo: (1) Start with the current best skill document s_k. (2) Run the CLM on benchmark tasks using s_k. (3) Collect trajectories, accuracy, and computational costs. (4) Have the proposer LLM examine those results and generate candidate improvements. (5) Evaluate the candidates on development tasks (Appendix E pg 23 In-context evolution: on 6 training instances, if pass then 12 training, if pass then 102 development set). (6) Select the best candidate as s_{k+1}. Goto (2).

Evolving CLMs with an optimization loop. CLMs can also be optimized through textual evolution. For task instance x, let $\tau(x; s)$ be the trajectory induced by Eq. 4, and $R(\tau)$ a trajectory-level reward. We optimize

Find max (best skill s*) by using expected reward across distribution of tasks x.

$$s^* = \arg\max_s \mathbb{E}_{x\sim\mathcal{D}}\left[R(\tau(x; s))\right], \quad (5)$$

Tau is entire agent trajectory for task x using skill document s

while keeping everything else fixed. In our implementation, we use a prompt-evolution loop (Agrawal et al., 2026): In each round, the agent produces rollouts on the training split, and a proposer model uses the resulting traces to generate candidate skills. We evaluate these candidates on the development split and select the skill for the next round. After evolution concludes, we evaluate the final selected skill once on the held-out test split. The optimizer may be either a stronger external model (*assisted evolution*) or the agent model itself (*self-evolution*), allowing context-management skills to evolve in context.

proposer and optimizer refer to same model

Reinforcement Learning for CLMs. CLMs can also learn context-management strategies through reinforcement learning and internalize them in model weights. Since context edits change the input across turns, we use stepwise GRPO (Shao et al., 2024). For each prompt, we sample a group of complete agent trajectories and compute the standard GRPO advantage from their trajectory-level outcome rewards. We then assign each trajectory's advantage to all of its constituent segments, so every model call is trained with the outcome of the full trajectory.

See App. E: 3040 deep research questions. Each step: 8 Q's, 32 traj so 256 traj/step. Temp 0.7 so stochastic traj.

Outcome rewards provide only weak supervision for context editing, as successful trajectories can contain inefficient edits and failed trajectories useful ones. Simply rewarding edit frequency or removed context volume is also undesirable, as the LM may reward-hack by making unnecessary edits that discard important information or hurt prefix reuse. We therefore introduce a **success-gated efficiency advantage** that further rewards successful trajectories with lower prefix-reuse FLOPs. Let c_i denote the prefix-reuse FLOPs of trajectory τ_i, and let $\mathcal{G}_g^+$ denote the successful trajectories in group g. We define $\bar{c}_g = \frac{1}{|\mathcal{G}_g^+|}\sum_{k\in\mathcal{G}_g^+} c_k$, and

$$A_i^{\text{eff}} = \begin{cases} \text{clip}\left(\frac{\bar{c}_g - c_i}{\bar{c}_g}, -1, 1\right), & i \in \mathcal{G}_g^+, \\ 0, & i \notin \mathcal{G}_g^+. \end{cases} \quad (6)$$

unsuccessful get zero

When there are fewer than two successful trajectories in a group, we set $A_i^{\text{eff}} = 0$ for all trajectories. Thus, the efficiency signal only re-ranks among successful trajectories by inference cost. We combine outcome and efficiency advantages as $A_i = A_i^{\text{out}} + w_{\text{eff}} A_i^{\text{eff}}$ to encourage trajectories that are both correct and efficient.

4.3 (More) Efficient CLM Serving with Suffix Cache Reuse

What if we want to serve CLMs even more efficiently and reduce the re-prefilling overhead? We introduce **Suffix Cache Reuse (SCR)**. As shown in Figure 4, when B is replaced by B' after an edit, SCR reuses the cached states of all surviving tokens, including C, and only reprefills the newly inserted or appended tokens B'. Surviving suffix tokens C thus retain stale cache states that encode the previous prefix, which can even be beneficial in some cases, as it retains richer information from the past. By contrast, standard prefix-cache reuse must re-prefill all tokens after the first mismatch (B' and C). [1] **Throughout the paper, we report prefix-reuse FLOPs under standard serving; additional SCR savings are reported separately in Section 5.3.**

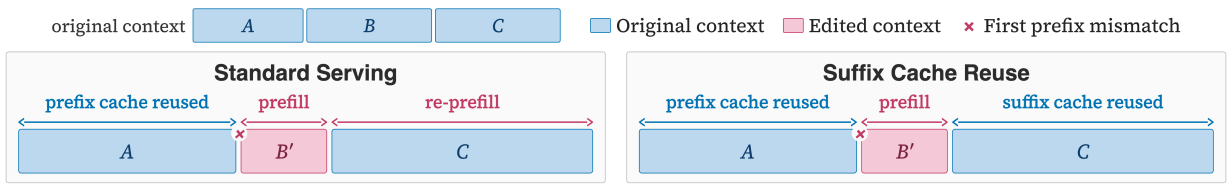


Figure 4 Comparison of standard serving and Suffix Cache Reuse (SCR). Standard serving reuses only prefix-matched cache, while SCR reuses cached states for all surviving tokens, reducing re-prefilling.

[1] In some LM chat-serving setups, such as Qwen3.6-27B's default chat template and GPT-5.6 Sol served through the stateless Chat Completions API, reasoning tokens from previous turns are stripped before the next turn, forcing the preserved suffix to be re-prefilled. SCR can also reuse the cached states of these preserved tokens.

Or it can be harmful because it shields C from B' changes, and if attention tends to focus on nearby tokens, then influence of B' will be reduced or blocked.

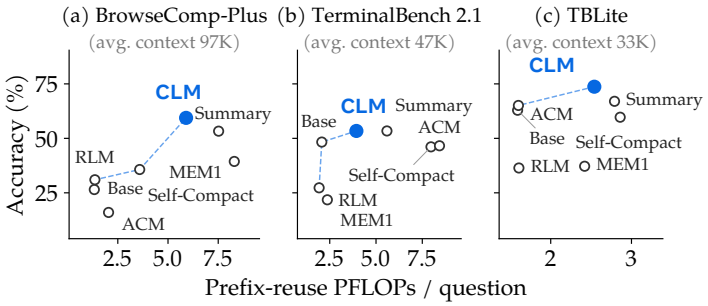


Figure 5 CLMs perform better than action-based and harness-defined baselines at lower cost on coding and deep research tasks. All methods use Qwen3.6-27B with a 32K context limit and a 100-turn cap. Blue dashed lines indicate the Pareto frontier.

Table 1 CLMs outperform specialized OpenEvolve evolutionary workflows on mathematical optimization problems. Best-of-run scores with Claude 4.6 Sonnet and a 32K context limit, capped at 100 scored attempts or five hours. Arrows indicate the direction of improvement; OE stands for OpenEvolve and SA for subagents.

Method	Circle packing (↑)	Heilbronn (↑)	Min-max/ min-dist (↑)	Erdős overlap (↓)
OE	2.541	0.03127	0.07690	0.38123
OE-Agent	2.525	0.03053	0.07724	0.38167
CLM	2.618	**0.03653**	**0.07758**	**0.38094**
CLM (SA)	**2.636**	0.03617	**0.07758**	0.38109

If we keep the underlying LLM, agent framework, and task environment essentially the same, does allowing the LLM to edit its own context improve task performance and computational efficiency?

5 Results

5.1 Evaluating CLMs Zero-Shot on Long-Horizon Agentic Tasks

We evaluate CLMs across long-horizon coding, deep research, and open discovery tasks, spanning tens to thousands of agent turns and runtimes from hours to a full day. Our evaluation covers both single- and multi-agent settings, including subagent and agent-swarm workloads for open discovery problems.

5.1.1 Coding and Deep Research Tasks

We first evaluate CLMs on two terminal-coding benchmarks, TerminalBench 2.1 (TB2.1) (Merrill et al., 2026) and TBLite (OpenThoughts-Agent team, 2026), and on the deep-research benchmark BrowseComp-Plus (BCP) (Chen et al., 2025).[2] We compare CLMs against MEM1 (Zhou et al., 2026), Self-Compact (Li et al., 2026b), ACM (Li et al., 2026c), and recursive language models (RLM) (Zhang et al., 2025a) with a shared Mini-SWE-Agent backbone (Merrill et al., 2026). To ensure a controlled comparison independent of training data, we evaluate all methods out of the box without training. We report performance and prefix-reuse FLOPs on these benchmarks for Qwen3.6-27B with a 32K context budget, with full details in Appendix E.

BCP

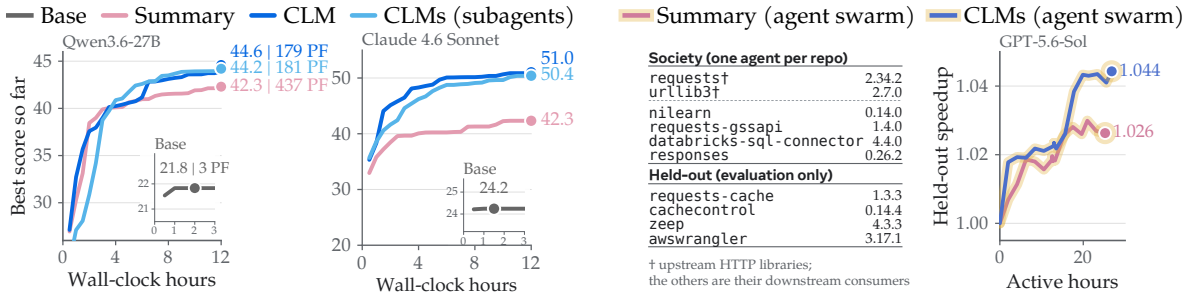
CLMs outperform harness-defined and action-based baselines. On BCP, CLMs outperform all baselines, scoring 59.4% at a 32K context limit and exceeding the strongest baseline, Codex-style summarization, by 11.4% relative. CLMs also use 21.5% and 28.9% fewer prefix-reuse FLOPs than the next two strongest methods, Codex-style summarization and MEM1, respectively. On coding benchmarks, CLMs match the strongest baseline, Codex-style summarization, on TB2.1 while using only 70% of its prefix-reuse FLOPs, and exceed it on TBLite (73.7% against 67.0%) with 91% of its FLOPs.

5.1.2 Open Discovery Problems

Open discovery problems provide longer horizons as our testbeds. We consider three types of open discovery problems with increasing horizons: (1) **Mathematical optimization:** four mathematical optimization problems used by AlphaEvolve (Novikov et al., 2025) and OpenEvolve (Sharma, 2025): circle packing, min-max/min-distance 2D, Erdős minimum overlap, and the Heilbronn triangle problem. (2) **Single-repository optimization:** ten EdgeBench (Zhu et al., 2026) tasks (EdgeBench-10; Appendix E), where the agent optimizes within a repository for up to 12 hours. (3) **Multi-repository optimization with agent swarms:** six repositories jointly optimized by multiple agents and evaluated on held-out downstream packages. Runs last over 24 hours.

Mathematical optimization: CLMs vs. specialized evolutionary workflows. On mathematical optimization, we compare against OpenEvolve (Sharma, 2025), a specialized AlphaEvolve-style (Novikov et al., 2025) workflow for program generation, evaluation, and evolutionary selection. We also include OpenEvolve-Agent, which

[2]Deep research requires search tools. Instead of training the agent against a fixed tool interface, we expose the search tools as in-context skills. This follows our less-is-more design principle: we keep as little as possible hard-coded at the harness level, so that tools can be flexibly defined, added, or revised at inference time.



(a) EdgeBench-10 single-repository optimization.

(b) Software World multi-repository optimization.

Figure 6 CLMs outperform Codex-style summary harness on long-horizon repository optimization. (a) EdgeBench-10 with a 32K context budget. Curves show best-of-three scores over 12 hours for Qwen3.6-27B and Claude 4.6 Sonnet; end labels show final scores and, for Qwen3.6-27B, mean compute per trial (PF = prefix-reuse PFLOPs). (b) Software World with GPT-5.6-Sol and a 272K context budget. Six agents jointly optimize interdependent repositories and are evaluated on four unseen downstream packages; the right panel shows geometric-mean speedup over 17 evaluation tasks.

replaces its proposer with a Mini-SWE-Agent that can interact with the environment before each submission. For CLM, we use the same base harness with a minimal Bash interface and provide the evolutionary algorithm as in-context guidance, leaving planning and context management to the agent. Using Claude 4.6 Sonnet and the same evaluator, CLM achieves the highest best-of-run score on all four problems (Table 1; progress curves in Figure 18). This shows that a general agent with direct context control can outperform a specialized evolutionary workflow with less fixed orchestration.

Single-repository optimization under single-agent and subagent settings. On EdgeBench-10, agents optimize a repository for up to 12 hours with verifier feedback; we report the best score over three seeds per task. Figure 6 compares the base harness, Codex-style summarization, CLM, and CLMs with up to five concurrent subagents under a 32K context budget. With Qwen3.6-27B, CLM reaches 44.6 using 179 prefix-reuse PFLOPs per trial, versus 42.3 and 437 PFLOPs for summarization; the subagent variant reaches 44.2 at 181 PFLOPs. With Claude 4.6 Sonnet, CLM and its subagent variant reach 51.0 and 50.4, compared with 42.3 for summarization. We find that subagents provide little additional benefit on this single-repository benchmark.

Multi-repository optimization with agent swarms. We evaluate CLM on `Software World`, where six agents jointly optimize interdependent Python repositories and are evaluated on four unseen downstream packages (Figure 6b, left). This provides an extrinsic test of whether improvements transfer beyond the repositories the agents directly observe. Compared with a summary-based agent swarm at the same spend, CLM achieves 65% greater downstream speedup over the initial releases (Figure 6b, right). Full setup and scoring details are provided in Appendix E.

5.2 Learning Better Context-Management Strategies in Context or in Weights

CLMs make context management an intrinsic model behavior that can be learned like other skills. In this section, we present in-context learning and reinforcement learning results for CLMs.

Steering context management by simply talking to CLMs. Users can steer CLMs toward a desired context-management strategy through natural-language instructions. We demonstrate this with three behaviors: triggering compaction at a specified context length, compacting around semantic sub-question boundaries, and backing up the context before compaction. Each behavior is induced by a single sentence appended to the task prompt. As shown in Figure 7, the agent adapts its context-management policy accordingly, without any change to the harness or model parameters. Measurement details are provided in Appendix E.

Evolving context management via textual evolution with CLMs. We apply the in-context evolution loop from Section 4.2 to ContextBench with a 32K context budget. In **assisted evolution**, Qwen3.6-27B starts without any context-management instruction, with Claude Fable 5.1 serving as the skill proposer; in **self-evolution**, Opus 5 serves as both the agent and the proposer. Figure 8 shows results on KV Store from ContextBench, where both settings improve over their initialization and expand the performance–cost Pareto frontier, with evolved skills that can strictly dominate the starting point. Additional results are in Appendix F.

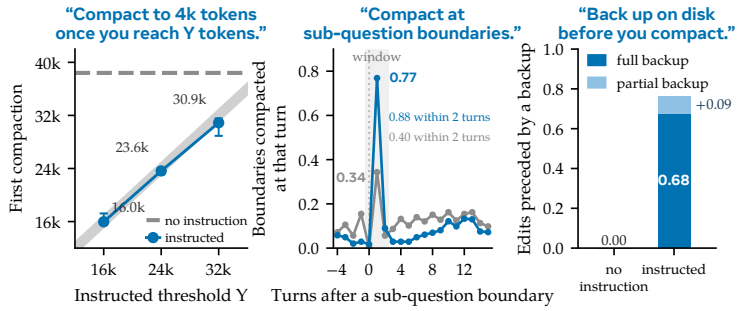


Figure 7 One sentence in the prompt changes the context-management policy. Natural-language instructions steer compaction timing, semantic boundaries, and backup behavior. Gray denotes no instruction and blue the instructed setting; exact prompts are in Appendix E.

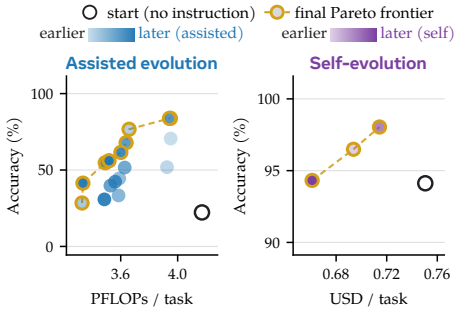


Figure 8 Textual evolution for CLMs on KV Store (32K budget). Assisted evolution uses Qwen3.6-27B with Claude Fable 5.1 as proposer; self-evolution uses Opus 5 for both roles.

Why Qwen-9 here instead of Qwen-27B? Better results?

Reinforcement learning for CLMs. We post-train Qwen3.5-9B on OpenResearcher using the reward formulation from Section 4.2 and evaluate on held-out BrowseComp-Plus. Before training, CLM with Qwen3.5-9B underperforms the summary harness by six points due to the smaller model's limited context-management capabilities. After RL, it gains 13.7 points to 42.5%, matching the trained summary harness while using 1.34 versus 2.19 PFLOPs per question. Adding the efficiency reward further reduces inference cost without a clear loss in accuracy for either CLM or the summary harness. Full results and reward ablations are provided in Appendix F.

Method	Acc. (%) ↑	PFLOPs / Q ↓
Summary	34.7 → 42.1	4.01 → 2.19
CLM	28.8 → **42.5**	1.52 → **1.34**

Table 2 RL results on BrowseComp-Plus with Qwen3.5-9B. Performance before and after training on OpenResearcher. The RL checkpoint is selected on a held-out validation set.

5.3 (More) Efficient Serving with Suffix Cache Reuse

As shown in Figure 9, SCR effectively reduces cache re-prefilling, matching the standard SGLang serving with 65.0% of its empirical prefix-reuse FLOPs on BCP. In addition, SCR is not limited to CLMs. Serving engines commonly strip prior reasoning tokens from chat histories, causing subsequent preserved tokens to be re-prefilled. We show that SCR can also reduce this re-prefilling cost in this more general setting.

We provide further details in Appendix B, including SCR implementation for hybrid models with interleaved full- and linear-attention layers, handling of multiple surviving post-edit spans, a decomposition of savings from reasoning-token stripping, and remaining opportunities for improvement in SGLang serving with SCR.

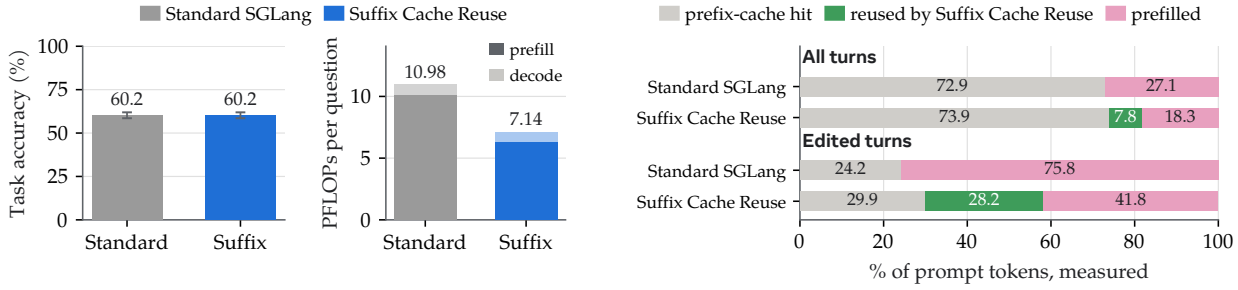


Figure 9 Comparison of Suffix Cache Reuse and standard SGLang serving on BCP with Qwen3.6-27B. *Left*: Task accuracy and prefix-reuse FLOPs per question. *Right*: Server-side compute decomposition, showing the fraction of prompt tokens by compute type across all turns and turns following context edits.

* Primary value shown by paper seems to be precise control over context compression. Context mgmt. as memory mgmt.
* Using this as a scratchpad memory seems like overkill since it is a lot of complexity (e.g. hacking SGlang, more prefix computation, living with less accurate 'C" suffix) for something that could equally well be done with an outside file.
* Paper does not establish that self-editing context is reliable.

Safety implications of a model-editable context. Granting models write access to their live context enables more flexible on-the-fly context management, but also creates new safety challenges. Editable context can become another channel through which prompt injections or self-generated instructions persist across turns. Recent work has observed such behavior in compaction summaries, including cases where a model inserted unauthorized instructions into its own summary that subsequently affected task behavior (OpenAI, 2026b). As editable context becomes more widely used, future work should characterize these new attack surfaces and develop defenses that preserve the flexibility of model-controlled context while maintaining its integrity.

Future directions: scaling CLM RL and distilling existing harnesses into CLMs. Future work can scale RL training so that CLMs can explore and learn effective context-management strategies, and develop a harness-to-CLM pipeline that distills strategies from existing harnesses into CLMs. This is motivated by the view that, while standard LMs only map input tokens to next-token distributions, harnesses determine how the context is constructed and updated. Since many harness operations can be expressed as context transformations, they can potentially be translated into CLM actions and eventually internalized into model weights. From this perspective, harnesses act as a form of procedural memory or task-specific skill that can be developed externally and later absorbed by CLMs for more general use.

Acknowledgments

We thank Sewon Min and Steven Zijian Chen for helpful discussions. We thank Ilia Kulikov and Mickel Liu for their help with infrastructure questions. This work was supported by the Singapore National Research Foundation and the National AI Group in the Singapore Ministry of Digital Development and Information under the AI Visiting Professorship Programme (award number AIVP-2024-001) and the AI2050 program at Schmidt Sciences.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 8479–8565, 2026.

Daman Arora and Andrea Zanette. Training language models to reason efficiently. *Advances in Neural Information Processing Systems*, 38:60770–60808, 2026.

Federico Cassano and Sasha Rush. Training Composer for longer horizons. Cursor Research Blog, March 2026. https://cursor.com/blog/self-summarization. Published March 17, 2026; accessed July 29, 2026.

Aaron Chan, Ahmed Shalaby, Alexander Wettig, Aman Sanger, Andrew Zhai, Anurag Ajay, Ashvin Nair, Charlie Snell, Chen Lu, Chen Shen, et al. Composer 2 technical report. *arXiv e-prints*, pages arXiv–2603, 2026.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, et al. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*, 2025.

In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, 2024.

Zhenyu He, Jun Zhang, Shengjie Luo, Jingjing Xu, Zhi Zhang, and Di He. Let the code llm edit itself when you edit the code. In *International Conference on Learning Representations*, volume 2025, pages 59637–59653, 2025.

Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 24391–24402, 2025.

Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyan Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. Memento: Teaching llms to manage their own context. *arXiv preprint arXiv:2604.09852*, 2026.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Mo Li, LH Xu, Qitai Tan, Long Ma, Hongyong Song, Ting Cao, and Yunxin Liu. Sculptor: Empowering llms with cognitive agency via active context management. In *International Conference on Learning Representations*, volume 2026, pages 153411–153440, 2026a.

Tianjian Li, Jingyu Zhang, William Jurayj, Xi Wang, Chuanyang Jin, Mehrdad Farajtabar, Eric Nalisnick, and Daniel Khashabi. Self-compacting language model agents. *arXiv preprint arXiv:2606.23525*, 2026b.

Xiaochuan Li, Ryan Ming, Meng Chu, Shuai Shao, Rong Jin, and Chenyan Xiong. Acm: Agentic context management for long horizon tasks. *arXiv preprint arXiv:2607.23809*, 2026c.

Yujiang Li, Zhenyu Hou, Yi Jing, Jie Tang, and Yuxiao Dong. CompactionRL: Reinforcement learning with context compaction for long-horizon agents. *arXiv preprint arXiv:2607.05378*, 2026d.

Shukai Liu, Bo Jiang, Jian Yang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon swe-agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 20604–20617, 2026.

Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. Codex CLI: A coding agent for the terminal. https://github.com/openai/codex, 2026a. Software, version 0.146.0, accessed July 29, 2026.

OpenAI. Self-generated prompt injections in compaction summaries, September 2026b. https://alignment.openai.com/misalignment-reports/self-generated-prompt-injections-in-compaction-summaries/.

Bespoke Labs OpenThoughts-Agent team, Snorkel AI. OpenThoughts-TBLite: A High-Signal Benchmark for Iterating on Terminal Agents. https://www.openthoughts.ai/blog/openthoughts-tblite, February 2026.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Keqin Peng, Yuanxin Ouyang, Xuebo Liu, Zhiliang Tian, Ruijian Han, Yancheng Yuan, and Liang Ding. Think dense, not long: Dynamic decoupled conditional advantage for efficient reasoning. *arXiv preprint arXiv:2602.02099*, 2026.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. https://github.com/algorithmicsuperintelligence/openevolve.

Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.

Richard S. Sutton. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019.

Shengguang Wu, Hao Zhu, Yuhui Zhang, Xiaohan Wang, and Serena Yeung-Levy. Automem: Automated learning of memory as a cognitive skill. *arXiv preprint arXiv:2607.01224*, 2026.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12805–12825, 2026.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. https://arxiv.org/abs/2405.15793.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 94–109, 2025.

Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, et al. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*, 2025.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.

Alex L Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025a.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025b.

Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026a.

Xuan Zhang, Longtao Zheng, Cunxiao Du, Bo An, and Xin Dong. Autocompact: Learning when to compact context in long-horizon coding agents, 2026b. https://autocompact.github.io/.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. In *International Conference on Learning Representations*, volume 2026, pages 58413–58438, 2026.

Deyao Zhu, Xin Zhou, Shengling Qin, Xuekai Zhu, Hangliang Ding, Shu Zhong, Zixin Wen, Zhonglin Xie, Chenhui Gou, Linxuan Ren, et al. Edgebench: Unveiling scaling laws of learning from real-world environments. *arXiv preprint arXiv:2607.05155*, 2026.

Appendix

A Extended Related Work

Harness-scheduled context management. A common approach is to let the harness determine when and how the context is updated. Systems such as Cursor (Cassano and Rush, 2026), Codex (OpenAI, 2026a), and Terminus2 (Merrill et al., 2026) trigger compaction when the context reaches a predefined length, using a prescribed summarization procedure. MEM1 (Zhou et al., 2026) instead updates the context at every turn, combining information retained from previous turns with the new observation rather than carrying forward the full history. Reinforcement learning can improve model performance within these harness-scheduled procedures: Composer (Cassano and Rush, 2026; Chan et al., 2026), CompactionRL (Li et al., 2026d), and MEM1 train models to preserve useful information or continue reasoning effectively under context compaction. Although the resulting context depends on the model's generation, the update schedule and procedure remain prescribed by the harness.

Model control within a constrained action space. Another line of work gives the model control over context management through constrained tools that implement predefined strategies, such as compaction, offloading, retrieval, and branching. Self-Compact (Li et al., 2026b) and AutoCompact (Zhang et al., 2026b) let the model decide when to compact. Context-as-a-Tool (Liu et al., 2026) exposes model-triggered compaction within a structured context workspace, and ACM (Li et al., 2026c) adds offloading and retrieval. AgeMem (Yu et al., 2026) combines long-term memory operations with tools for summarizing and filtering the current context. Context Folding (Sun et al., 2025) lets the model branch into a sub-trajectory and fold it into a summary upon returning, whereas AgentFold (Ye et al., 2025) condenses recent interactions or consolidates multiple historical steps through folding directives. Sculptor (Li et al., 2026a) supports fragment-level summarization, hiding, restoration, and search while preserving message count and order. Training can improve how models use these tools: AgentFold uses supervised fine-tuning, while AutoCompact, AgeMem, and Sculptor use reinforcement learning to optimize their respective context-management decisions. However, the available operations and their underlying strategies remain predefined by the tool interfaces. In contrast, CLMs treat *context as a file*, giving the model direct read and write access to its live context through general-purpose programming tools.

Model-controlled context through meta optimization. Meta-optimization gives models control over context management by improving the reusable procedures that govern agent execution. Meta-Harness (Lee et al., 2026) and AutoMem (Wu et al., 2026) optimize harnesses or memory-management procedures from trajectory feedback, while Meta Context Engineering (Ye et al., 2026) co-evolves context-engineering skills and context-construction functions represented as files and code. Related approaches optimize the agent program itself (Zhang et al., 2025b, 2026a) or evolve prompts through reflection on rollouts (Agrawal et al., 2026). From the perspective of CLMs, a harness encodes reusable procedures for managing context. Such procedures can also be expressed as skills that guide the model in editing its live context. Our skill evolution therefore shares the goal of harness optimization: improving reusable context-management procedures through evaluation feedback.

Context as a variable in the environment. Recursive language models (RLMs) (Zhang et al., 2025a) place a long input in a REPL variable that the model can access programmatically and process through recursive calls. This gives the model control over how it reads the input, but does not expose its own live context for direct editing. The distinction is twofold: RLMs externalize the input rather than the evolving interaction history, and the model's access to its live context remains read-only rather than read–write. CLMs instead make the live context itself editable, including information accumulated during execution. The two approaches are complementary: RLM-style access can keep large inputs outside the context until needed, while CLMs can manage the information brought into the context and the history generated while processing it.

Non-prefix KV cache reuse. With standard prefix caching, changing an early part of a prompt forces the serving system to recompute the KV states of everything that follows, even when the later text is unchanged. Prior

work relaxes this requirement in different settings. Prompt Cache (Gim et al., 2024) precomputes attention states for predefined prompt modules, allowing a module to be reused in prompts that do not share the same preceding text. In retrieval-augmented generation, the same document may appear after different documents or instructions. CacheBlend (Yao et al., 2025) and EPIC (Hu et al., 2025) reuse cached document chunks in these new contexts, recomputing selected tokens to account for the changed surroundings. PIE (He et al., 2025) studies cache reuse when a user modifies previously processed code and requests a new completion. It retains cached states for unchanged text after an edit and corrects their rotary positions, avoiding suffix recomputation. Memento (Kontonis et al., 2026) evicts each completed reasoning block from the KV cache but keeps the cached states of its summary, which were computed while the block was still in context, and finds that these states retain useful information from the evicted block. Suffix Cache Reuse applies the same reuse principle to an agent's live context: when the agent replaces a span, the unchanged suffix retains its cached states rather than being prefilled again. We integrate this mechanism into SGLang for agent-driven context editing and further extend it to hybrid architectures that combine full-attention layers with linear-attention layers.

Reinforcement learning for context management. Context edits break the append-only structure of an agent trajectory: the final context may no longer contain the inputs under which earlier actions were generated. Training must therefore preserve these intermediate contexts and evaluate each generated segment under its original input. ReSum (Wu et al., 2025) segments trajectories at summarization boundaries and broadcasts the trajectory-level advantage to all segments. Sculptor (Li et al., 2026a) similarly preserves training contexts around context modifications and masks previously trained completions to avoid counting them repeatedly. Our stepwise GRPO follows this principle, assigning the final outcome advantage to each segment's policy-gradient loss rather than differentiating through the context edits. Prior work also considers efficiency. AgeMem (Yu et al., 2026) includes a reward for context compactness, while Sculptor penalizes exceeding tool-call or trajectory-length budgets. Our efficiency signal instead measures trajectory-level inference FLOPs under prefix caching, accounting for the recomputation that context edits can incur. We add a success-gated efficiency advantage that favors lower-cost trajectories only within the successful subset of a rollout group. This distinguishes reducing inference compute from merely shortening the context and avoids giving failed trajectories an efficiency bonus.

Success-conditioned efficiency objectives. Prior work has also conditioned efficiency rewards on task success. Arora and Zanette (2026) penalize response length only for correct answers, and DDCA (Peng et al., 2026) computes a separate length advantage within the correct-response subset. These objectives primarily measure efficiency through response or trajectory length. Our formulation instead uses trajectory-level FLOPs under prefix caching, capturing the recomputation induced by context edits in addition to generated length.

Context management and external memory. Context management determines what the model sees at each invocation, whereas external memory stores information beyond the current context for later use. Memory-R1 (Yan et al., 2026), for example, learns to manage stored memories and use retrieved information. These mechanisms work together: an agent can offload information to reduce its context and retrieve it when needed. MemGPT (Packer et al., 2023) connects them through a memory hierarchy, allowing the model to edit a designated, fixed-size block within the context rather than the entire live context. AgeMem (Yu et al., 2026) jointly learns external-memory operations and tools that summarize or filter the current context. This distinction depends on the role of the information, not its storage format. In CLMs, the context file specifies the input to subsequent model calls. Other files can serve as external memory, with their contents entering the context when retrieved. Making the live context editable therefore complements external memory by letting the model decide how retrieved information is incorporated and when it is removed.

B Suffix Cache Reuse

Background: radix tree and prefix cache reuse in SGLang. SGLang (Zheng et al., 2024) keeps the KV cache of served requests in a radix tree over token sequences: each edge holds a token span and the KV entries computed for it, so requests with a common prefix share one path. A new request walks the tree to its longest matching prefix, reuses the KV entries along that path, and prefills only the remaining tokens; unused nodes

ChatGPT insert:

A **radix tree** is a data structure for efficiently storing and searching sequences that share common prefixes. SGLang uses one to organize previously computed **KV-cache entries**, so that when a new LLM request arrives, it can reuse the computations for the longest prefix it has already processed.

The key idea is that **the tree is indexed by token sequences, not by the numerical values of the K and V vectors themselves.**

I'll first explain an ordinary radix tree, then show how SGLang applies it to transformer inference.

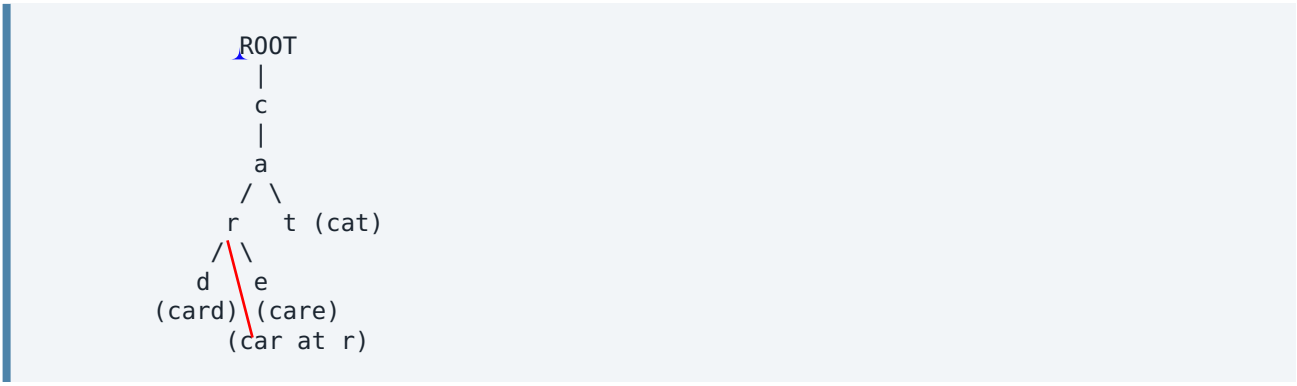
1. What is a radix tree?

Start with a *trie*, also called a prefix tree.

Suppose we want to store four words:

- `car`
- `card`
- `care`
- `cat`

An ordinary trie stores one character per edge:



A conventional trie. Each edge represents one character, and common prefixes share the same path.

The first two characters `ca` are stored only once because all four words share them.

A **radix tree**, or compressed trie, takes this further: any unbranching chain of edges can be compressed into one edge containing multiple characters.

```
              ROOT
               |
              ca
             /  \
            r    t (cat)
           / \
          d   e
       (card) (care)
         (car at r)
```

A radix tree compresses the unbranched `c → a` path into one `ca` edge.

This reduces the number of nodes and pointers needed, particularly when sequences have long common prefixes.

For LLMs, an edge might represent **thousands of tokens**, not just two characters.

2. How SGLang uses a radix tree

Suppose SGLang processes these three LLM requests:

Request 1

`[System prompt] [User: Explain attention]`

Request 2

`[System prompt] [User: Explain convolution]`

Request 3

`[System prompt] [User: Explain attention] [Assistant reply] [User follow-up]`

All three requests share the same system prompt. Requests 1 and 3 share an even longer prefix.

The radix tree can represent them as:

```
                    ROOT
                     |
              [System prompt]
                /          \
               /            \
[Explain attention]    [Explain convolution]
         |
   [Assistant reply]
         |
    [Follow-up]
```

Each edge is associated with:

1. A sequence of token IDs.
2. References to the previously computed KV-cache entries for those tokens.

Crucially, SGLang does not need to duplicate the system prompt's cached KV states for each request.

The shared prefix corresponds to shared cached computations.

3. What exactly is stored?

Conceptually, an edge contains something like:

```
edge = {
    "token_ids": [15496, 11, 703, 389, ...],
    "kv_slots": [100, 101, 102, 103, ...]
}
```

This is illustrative, not the literal SGLang data structure.

The radix tree's metadata generally resides on the CPU, while the KV tensors occupy allocated memory in the GPU's cache pool.

The edge stores references to the KV locations, not necessarily the full KV tensors inside the tree node itself.

For every token position and conventional full-attention layer, the model has calculated key and value representations.

`K_i = W_K h_i, V_i = W_V h_i`

Because a causal transformer's representation at position i depends on the preceding tokens, the cached K_i,V_i are valid for reuse when the preceding token sequence is identical.

This is why prefix sharing is exact for ordinary causal attention with consistent positions and model configuration.

4. How longest-prefix matching works

Consider three previously processed token sequences:

```
[A B C D E]
[A B C F G]
[A B H I]
```

The radix tree organizes them as:

```
ROOT
 |
[A B]
  |
  +---- [C]
  |      |
  |      +---- [D E]
  |      |
  |      +---- [F G]
  |
  +---- [H I]
```

Now a new request arrives:

```
[A B C D X Y]
```

SGLang traverses the tree:

Step	Tokens checked	Result
1	A B	Match
2	C	Match
3	D	Match
4	E versus X	Mismatch
5	Stop	Reuse A B C D

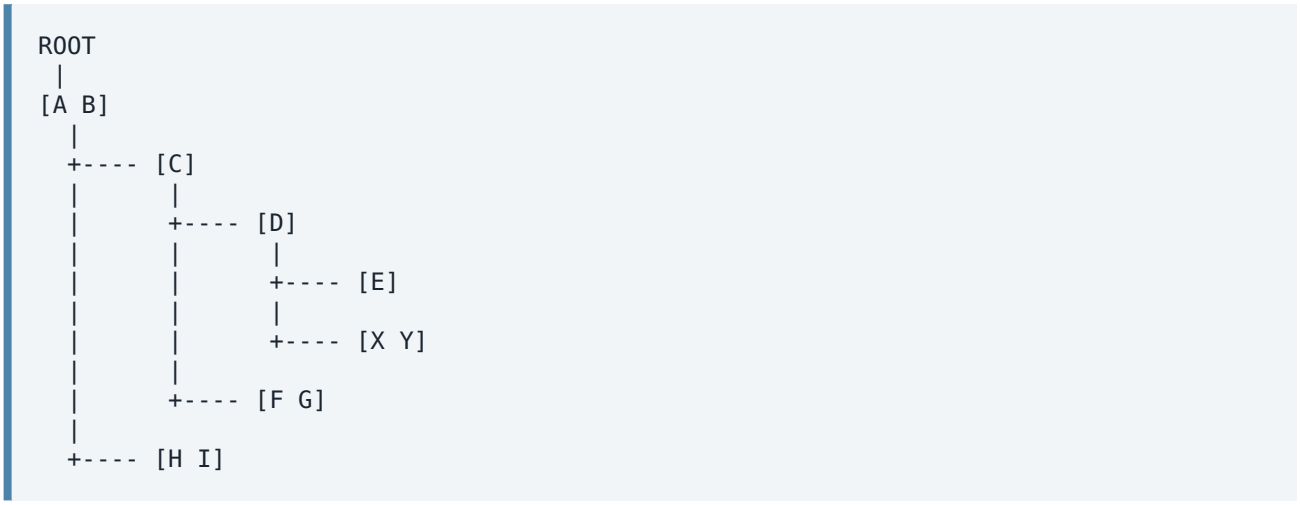
The server therefore processes only:

```
[X Y]
```

rather than recomputing all six tokens.

Strictly, it also needs to split the existing `[D E]` edge into `[D]` and `[E]` to represent the new branch.

The resulting tree is:



The radix tree can therefore grow dynamically as new sequences are processed.

One nuance: real SGLang configurations may match in cache-page units, rather than sharing at every single token boundary. The diagrams above show the logical token-level behavior.

5. Why not use a hash table?

A hash table can tell you quickly whether a complete token sequence is already cached.

But what happens when only the first 8,000 tokens of a 12,000-token request are cached?

A simple full-sequence hash lookup would miss.

You could maintain hash entries for every possible prefix length, but that introduces more indexing and dependency-management complexity.

A radix tree naturally supports the three operations SGLang needs:

Operation	Radix tree behavior
Find longest matching prefix	Traverse edges until first mismatch
Insert a new request	Add a branch, splitting an edge if necessary
Evict cached data	Remove no-longer-needed leaf branches

In this application, the tree isn't used to accelerate the transformer's matrix multiplications themselves. Its purpose is to determine **which computations can be skipped entirely**.

6. How LRU eviction works

The KV cache is finite. GPU memory cannot retain the results of every request forever.

Suppose the cache tree contains:

```
                ROOT
                  |
                [A B]
               /     \
          [C D]       [E F]
            |
          [G H]
```

Assume `[G H]` was least recently used.

The eviction procedure can remove that leaf's KV entries and free the corresponding GPU memory.

After removing `[G H]`, `[C D]` may become an evictable leaf itself.

But SGLang must not evict `[A B]` while its descendants still depend on that prefix, nor evict nodes whose KV entries are actively referenced by running requests.

This is why the paper specifically refers to **least-recently-used eviction of unused nodes**.

LRU is a policy for deciding which eligible cached data to discard; the tree structure defines which branches can safely be discarded.

7. Why the radix tree matters for CLM

Now consider a CLM context consisting of:

```
[A B C]
```

The server has cached all three spans.

The model decides to edit B, producing:

```
[A B' C]
```

SGLang searches the radix tree for the longest prefix matching the new prompt.

It finds only A.

Although C's tokens are identical to their previous values, they are no longer at the end of an identical prefix.

Therefore, standard SGLang must recompute both B′ and C.

Standard radix-tree prefix reuse

`[A: Reuse cached KV] [B′: Recompute] [C: Recompute]`

With Suffix Cache Reuse (SCR)

"Session-private slot" is a KV cache slot that is NOT in the radix tree. They keep C out of tree because it no longer is perfect, given B changed.

```
[A: Reuse KV] [B′: Recompute] [C: Reuse KV]
```

The challenge is that C's cached representations were originally computed from `[A B C]`, not `[A B′ C]`.

Therefore, reusing C after changing B is approximate, not exact.

The authors' Suffix Cache Reuse implementation goes beyond the original radix-tree matching: it identifies surviving suffix spans, relocates their cached states into session-private slots, and adjusts rotary position encoding on keys where necessary.

Importantly, **SCR does not fundamentally replace the radix tree**. It retains the conventional prefix-sharing mechanism and adds another mechanism for reusing cached states beyond the first mismatch.

8. One subtlety for hybrid models like Qwen3.6-27B

The paper notes that Qwen3.6-27B has 64 layers, including 48 linear-attention layers.

Unlike conventional attention layers, those linear-attention layers do not necessarily maintain a separate K/V history for every token. Instead, they maintain a recurrent state.

This makes radix-tree caching more complicated.

In a conventional full-attention layer, the server can identify and reuse a span of per-token KV entries.

For a recurrent linear-attention layer, resuming at an arbitrary token boundary requires the recurrent state at that boundary.

If the server only saved a checkpoint at the end of a much longer token span, it may be unable to resume from the desired branch point.

This is why Appendix B mentions that hybrid-model prefix matching sometimes falls back to a shorter cached prefix than expected.

The authors suggest more frequent recurrent-state checkpoints, such as at message boundaries, as a possible improvement.

Bottom line: A radix tree is a compressed prefix tree that maps token sequences to reusable cached computations. SGLang uses it to find the longest previously processed prefix, reuse its KV cache, and compute only the unmatched suffix.

The tree is especially useful for multi-turn agents because successive prompts often share thousands of initial tokens.

But its fundamental limitation is that **prefix matching stops at the first changed token**. That is the precise limitation the CLM paper's Suffix Cache Reuse method attempts to overcome.

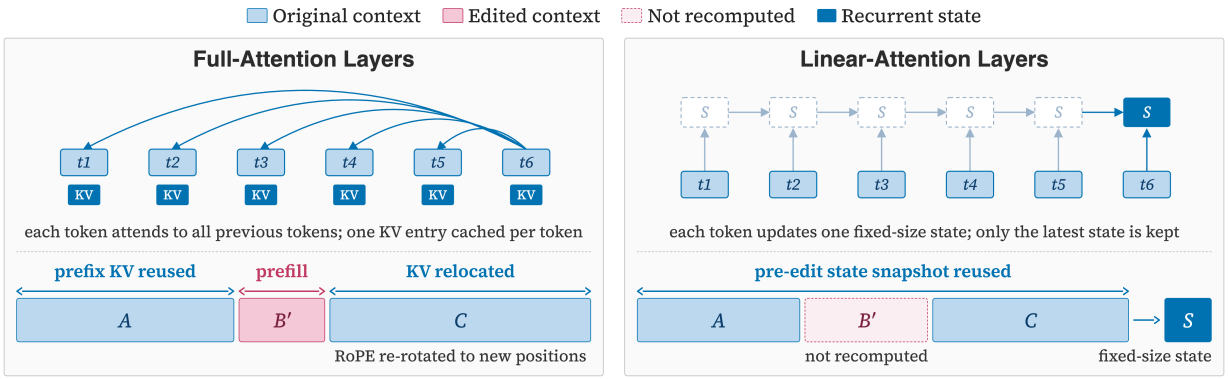


Figure 10 Suffix Cache Reuse for full attention layers and linear attention layers.

are evicted in least-recently-used order. Reuse therefore stops at the first mismatched token. This is exact for append-only histories, since each token's key and value depend on all preceding tokens and, through rotary position encodings, on its absolute position. After an in-the-middle edit, however, every token after the edit is re-prefilled, including text that survived unchanged. For hybrid models, whose linear-attention layers keep a fixed-size recurrent state instead of per-token entries, a match can resume only at a node that stores a state checkpoint. Suffix Cache Reuse keeps this tree as is and extends reuse to the surviving tokens beyond the matched prefix, as described next.

Suffix Cache Reuse implementation for standard full attention layers. We implement Suffix Cache Reuse as a patch to SGLang. As shown in Figure 10 (left), consider a context $[A\ B\ C]$ in which an edit replaces B with B'. Standard serving matches only A and re-prefills B' and C. When a new prompt arrives, Suffix Cache Reuse diffs it against the session's previous prompt to find the spans that survived the edit, and relocates up to K of them, largest first (K=6 in the main text). For each relocated span such as C, it reuses the cached keys and values, re-rotates the keys' rotary position encodings to their new positions, and splices them in after B'. Only B' and newly appended tokens are prefilled. Relocated entries live in session-private cache slots, so the shared radix tree never holds a moved entry; if these slots cannot be allocated, the server falls back to standard re-prefilling. Because the reused states were computed under the pre-edit context, Suffix Cache Reuse approximates re-prefilling, and K bounds the number of relocated spans per edit.

Suffix Cache Reuse implementation for linear-attention layers. In hybrid models such as Qwen3.6-27B[3], full-attention and linear-attention layers may be interleaved. As shown in Figure 10 (right), linear-attention layers maintain a fixed-size recurrent state rather than per-token caches, so there are no token-level entries to relocate. For these layers, we snapshot the recurrent state before the edit and continue from that snapshot, while the edit is reflected only in the 16 full-attention layers that retain token-level context. As a result, the newly inserted B' is not recomputed in the linear-attention layers, since subsequent tokens depend only on the reused recurrent state. Its representation is still recomputed in the full-attention layers and can influence later linear-attention layers through their inputs.

Suffix Cache Reuse for multiple surviving post-edit spans. A single edit may leave multiple surviving spans after the edit point. SCR can relocate each span, but every relocation reuses states computed under the pre-edit context and therefore introduces an approximation. When many spans are relocated in the same edit, these approximations can compound before the model has a chance to adapt in subsequent turns. We therefore cap the number of relocated spans per edit at K, reusing the K longest spans and re-prefilling the rest. This bounds the amount of approximation introduced at once and makes SCR less susceptible to pathological edits with many surviving spans.

We conduct a small-scale sensitivity analysis over $K \in \{1, 2, 3, 6, 12, 64\}$ on 64 BrowseComp-Plus questions with Qwen3.6-27B (Figure 11). Performance is robust across K, while cache-reuse gains largely saturate by

[3]Qwen3.6-27B is a hybrid model in which 48 of 64 layers use linear attention.

$K = 6$. We therefore use $K = 6$ throughout as a conservative choice that captures most reusable cache while limiting the approximation introduced by any single edit. In this small-scale analysis, varying K mainly affects cache-reuse efficiency, with no observed performance degradation.

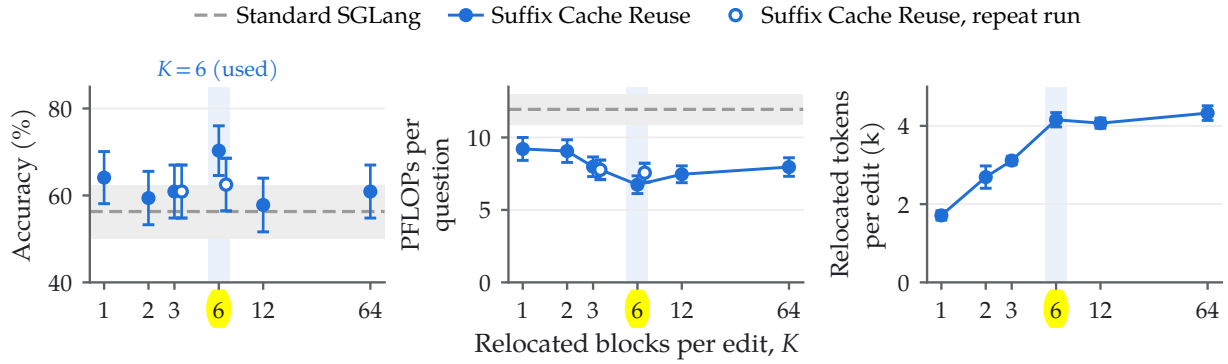


Figure 11 Small-scale sensitivity study of relocated spans per edit, K. Results on 64 BrowseComp-Plus questions with Qwen3.6-27B. Left: task accuracy. Middle: prefix-reuse PFLOPs per question. Right: relocated tokens per edited request. Dashed lines show standard SGLang; hollow markers show a repeat run; the shaded column marks K=6, used elsewhere. Error bars show ±1 standard error across questions; the gray band shows ±1 standard error for standard SGLang.

Surprising!

Bonus: Suffix Cache Reuse for stripped reasoning tokens in chat endpoints. Chat templates for reasoning models, including Qwen3.6, often remove the reasoning block from earlier assistant turns once the next user message arrives. Standard serving then re-prefills all preserved text after the first removed block, even when the agent never edits its own context. SCR treats reasoning stripping as another context edit and reuses the cached states of the preserved text, extending its benefit to standard chat serving.

Figure 12 shows that this effect accounts for a significant portion of SCR's savings on BrowseComp-Plus. Of the 7.8% of all prompt tokens reused by SCR beyond prefix-cache hits, 5.3 points come from reasoning stripping and only 2.5 from other context edits. Thus, a large fraction of SCR's benefit applies even to standard reasoning-model serving without model-driven context editing.

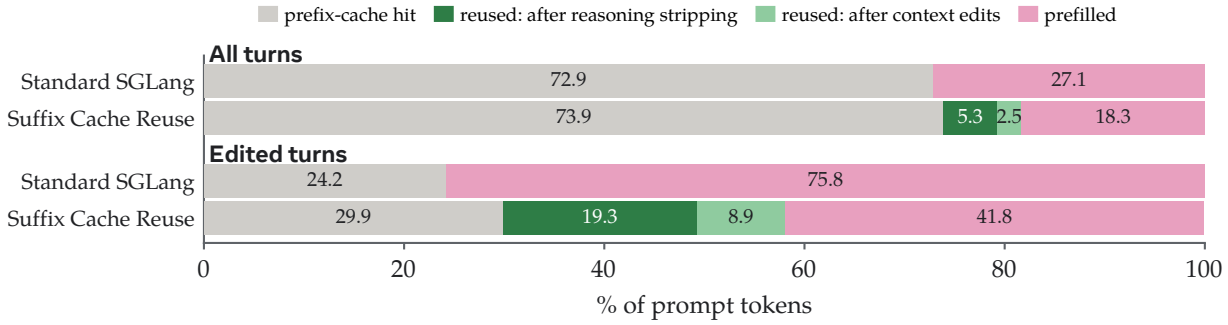


Figure 12 Suffix Cache Reuse also benefits standard chat serving. On BrowseComp-Plus, SCR reuses cached states not only after model-driven context edits but also after reasoning tokens are stripped from prior turns in standard chat endpoints. Results use Qwen3.6-27B on 830 questions with $K = 6$.

A common serving bottleneck and future improvement space. Figure 13 shows that SCR removes most re-prefilling of unchanged suffix tokens after an edit. Much of the remaining redundant prefill instead comes from unchanged prefixes that standard prefix caching should ideally reuse. This is a limitation of the current SGLang caching implementation for hybrid models, rather than SCR itself. Linear-attention layers maintain recurrent states, which SGLang stores only at cached request boundaries; when a later prompt diverges inside a cached span, no recurrent state is available near the branch point, so cache matching can fall back to a

much shorter prefix. This affects both standard prefix caching and the efficiency attainable with SCR. Storing recurrent states at finer-grained locations, such as message boundaries, is therefore a promising direction for improving both.

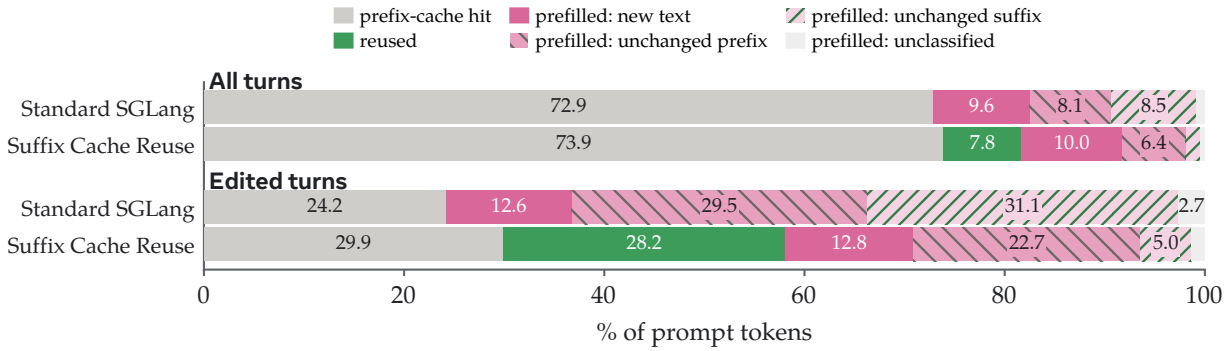


Figure 13 Remaining re-prefill under Suffix Cache Reuse on BrowseComp-Plus. SCR removes most re-prefilling of unchanged suffix tokens, while substantial unchanged-prefix prefill remains due to the current hybrid-model caching behavior in SGLang. Same runs as Figure 12.

C Prefix-Reuse FLOPs Computation

Prefix-reuse FLOPs measure the computation performed by a server with prefix caching over an agent trajectory. At turn t the prompt contains P_t tokens and the model generates G_t tokens. The server reuses the longest prefix of leading messages that also appeared in the prompt of an earlier turn, comprising R_t tokens, and prefills only the remaining $U_t = P_t - R_t$ tokens. Under standard prefix caching, an edit therefore invalidates the cached computation from the edited message onward, and a response is prefilled again when it first appears in a later prompt, in addition to being decoded when it is produced.

Consider a model with L layers of hidden size d and MLP width d_{ff}, of which L_{attn} are full-attention layers with h_q query heads (with an output gate), h_{kv} key–value heads and head dimension d_h, and L_{lin} are Gated DeltaNet layers with h_k key heads, h_v value heads, head dimensions d_k and d_v, and two scalar gates per value head. Linear operations have the same cost per processed token regardless of context length. Counting two FLOPs per multiply-add, this per-token cost is

$$C_{\text{token}} = \underbrace{6Ld\,d_{\text{ff}}}_{\text{MLP}} + \underbrace{L_{\text{attn}}\left[2d(2h_q d_h + 2h_{kv} d_h) + 2h_q d_h d\right]}_{\text{full-attention projections}} + \underbrace{L_{\text{lin}}\left[2d(2h_k d_k + 2h_v d_v + 2h_v) + 2h_v d_v d\right]}_{\text{Gated DeltaNet projections}}. \tag{7}$$

Only the full-attention layers incur a context-dependent cost. Each query–key pair costs one multiply-add per head dimension for the attention score and one for the weighted value, so the cost per pair is

$$C_{\text{attn}} = 4L_{\text{attn}} h_q d_h. \tag{8}$$

Ignoring lower-order boundary terms, a turn costs

$$F_t = C_{\text{token}}\,(U_t + G_t) + C_{\text{attn}}\left[\tfrac{1}{2}\left(P_t^2 - R_t^2\right) + G_t P_t + \tfrac{1}{2}G_t^2\right], \tag{9}$$

where the first term inside the brackets counts attention during prefill and the latter two count attention during decoding. The cost of a trajectory of T turns is $\sum_{t=1}^{T} F_t$. We omit the embedding and output layers, the Gated DeltaNet recurrent-state update, and normalization, activation and softmax operations.

Example: Qwen3.6-27B. Qwen3.6-27B has $L = 64$ layers with $d = 5120$ and $d_{\text{ff}} = 17{,}408$. Its $L_{\text{attn}} = 16$ full-attention layers have $h_q = 24$, $h_{kv} = 4$ and $d_h = 256$, and its $L_{\text{lin}} = 48$ Gated DeltaNet layers have $h_k = 16$, $h_v = 48$ and $d_k = d_v = 128$. Substituting these values gives $C_{\text{token}} = 48.70 \times 10^9$ FLOPs per token, of which the

MLP accounts for 34.23×10^9, the full-attention projections for 3.36×10^9 and the Gated DeltaNet projections for 11.12×10^9, and $C_{\text{attn}} = 3.93 \times 10^5$ FLOPs per query–key pair.

Figure 14 illustrates the computation saved by prefix caching within one turn. **The lengths are chosen for illustration and do not come from a particular run**: a prompt of $P_t = 20{,}000$ tokens and a response of $G_t = 500$ tokens, with the reusable prefix R_t set to 18,000, 10,000 or 0 tokens. When the turn only appends to its context, prefix caching avoids 87% of the computation the turn would need without a cache, and the turn costs 1.41×10^{14} FLOPs. An edit in the middle of the context leaves 10,000 reusable tokens and raises the cost to 5.74×10^{14} FLOPs. An edit at the start, equivalent here to having no reusable prefix, costs 10.81×10^{14} FLOPs, 7.7 times the append-only turn.

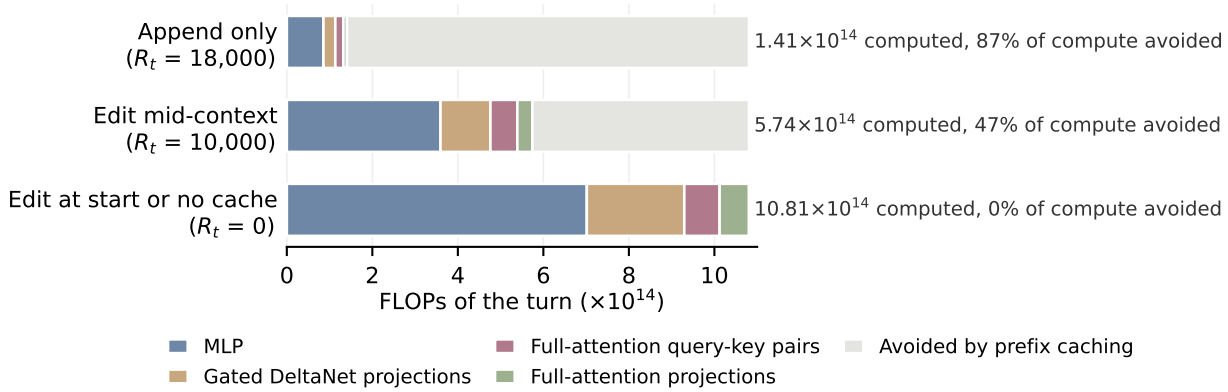


Figure 14 FLOPs of one Qwen3.6-27B turn for illustrative lengths ($P_t = 20{,}000$, $G_t = 500$) and three reusable-prefix lengths R_t. Colors split the FLOPs actually computed by operation; gray shows the additional computation that would be required without prefix caching.

D ContextBench

Design and examples of ContextBench tasks. Each ContextBench task grades what the agent kept, updated, or offloaded from an input that, at all but the lowest levels, exceeds the context window. The environment delivers the input as a sequence of operations, and each operation arrives as a new user message in one continuous conversation. An operation therefore enters the agent's context before the agent can act on it, and no harness can truncate or offload it on the way in. The agent controls only when the next operation arrives: it runs `echo READY_FOR_NEXT_OP` once it has handled the current one. The tasks need no search or inference beyond following the instruction, so an agent that could keep everything would answer every query; performance measures how the agent manages its context and nothing else. Every task instance is generated from a seed and checked before use. Of the 32,768-token context limit, 2,048 tokens are reserved for the model's response, leaving a usable budget of 30,720 tokens; a single operation must fit in a fifth of it and everything the task requires the agent to retain in half of it, each with a 10% margin, so a failure at any level reflects how the context was managed rather than a task that cannot be solved within the budget. Table 3 summarizes the four tasks and Figure 16 shows operations from each.

Context pressure is the total input the environment pushes over an episode divided by the 32,768-token context limit, so 1× is the point where an agent that keeps everything would fill its context. Each task reaches higher pressure through one generator setting, with everything else fixed (Figure 15); levels below 1× are included as controls on which keeping everything still fits.

Detailed instructions for isolated context-management diagnostics. In our pilot study, we provide each harness with a detailed task instruction and a method-specific skill that explains how to use its available tools for context management on that task. This helps isolate context-management capability from differences in task understanding or tool use. For each task, every method receives the same task instruction, followed by a skill tailored to its context-management mechanism. The task instruction specifies the input stream,

Task	Input stream	Queries	Metric
Needle Retention	~4K-token chunks, each with 2–8 needle lines and 140 filler lines	none	needle lines retained verbatim in the final context
Sudoku Sketchpad	one move per turn on a 16×16 board	current board after each move	board versions reproduced exactly
KV Store	batches of 100 SET operations with random 24-word values	24 GET queries	exact-value accuracy
Log Triage	batches of 14–54 service log lines	24 lookup and count queries	exact-answer accuracy

Table 3 The four ContextBench tasks. All metrics are computed from the agent's context; answers held only in files are not credited.

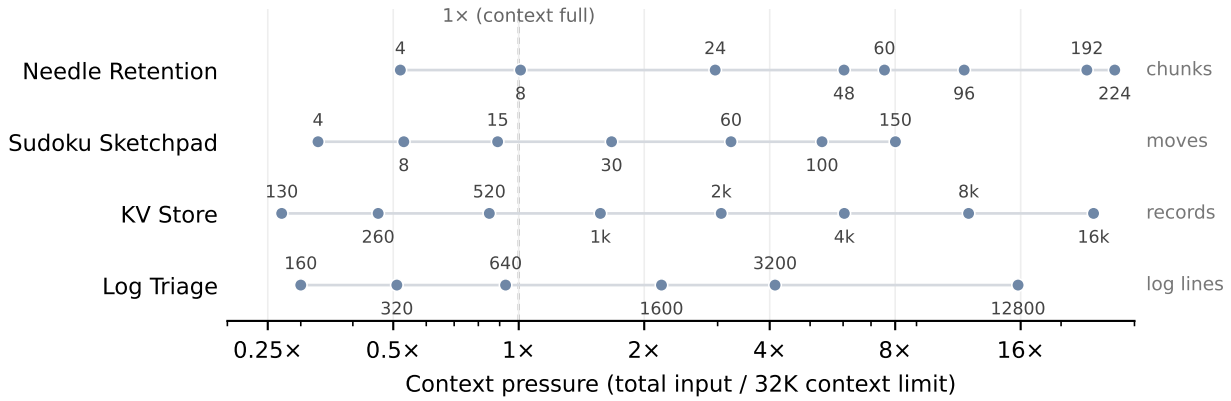


Figure 15 Context pressure of every ContextBench level. Each row varies one generator setting (unit at right); the label on each point is its value. Total input is every message the environment pushes over the episode, including the task instruction and, for Sudoku, the board once per move, counted in o200k tokens; pressure is total input divided by 32,768.

turn protocol, answer format, and grading procedure, without giving context-management advice. The method skill explains how to manage context using the tools exposed by that harness, including the relevant commands or tool calls, when they take effect, how to verify them, and common failure modes. We write each skill to a level of detail comparable to the CLM skill, so that baseline performance reflects what the harness enables rather than what the model must infer about how to use it. Table 4 shows excerpts of each method's KV Store skill. Skills are inserted at the same position relative to the task instruction for all methods, and all are released with the benchmark.

E Experimental Configurations

We use the original baseline implementations when available and run all methods with the model and budget specified for each experiment: the summary harness uses the Codex summarization prompts and compacts at 75% of the budget; Self-Compact uses the original prompts and self-check rubric and asks the model every two turns whether to compress once the context exceeds 37% of the budget; RLM runs its released harness; ACM runs its released code, in which the model decides when to manage its context. MEM1 is our re-implementation of its inference loop. CLM receives its editing reminder 2,048 tokens before the budget[4], and the base harness has no trigger. Unless stated otherwise, token budgets are counted with the o200k tokenizer and all methods are evaluated with the same base LM.

ContextBench. Every method uses GPT-5.4 through the API with a 32,768-token context limit, of which 2,048 tokens are reserved for the response, and receives the skill for its harness (Appendix D). Each level is

[4]In Appendix G, we analyze context-length awareness in existing LMs and find that it degrades at long context lengths. Simple environmental hints substantially improve estimation, so we use them as a temporary augmentation. Future models may acquire stronger context awareness directly or estimate usage on demand through context tools.

Needle Retention (first of 4 chunks)

```
=== chunk 1/4 (3 op(s) remaining after this) ===
NEEDLES (keep these lines verbatim in your context):
[n00000i00#f56502a0] glacier orbit lantern lattice quartz ember ember meadow lattice orbit.
...
<<<FILLER-BLOCK 00000#d4aff96b START -- delete this entire block (through its END line) from your context this turn>>>
[f00000x000#...] ... (140 filler lines)
<<<FILLER-BLOCK 00000#d4aff96b END>>>
```

Sudoku Sketchpad (initial board, first move, and expected panel)

```
Your STARTING sketchpad (version 0):
<<<SKETCHPAD BEGIN>>>
VERSION: 0
(1,1,.) (1,2,8) (1,3,.) (1,4,G) (1,5,6) (1,6,7) (1,7,C) (1,8,.) ... (1,14,E) (1,15,.) (1,16,4)
(2,1,.) (2,2,.) (2,3,.) (2,4,E) (2,5,.) (2,6,.) (2,7,B) (2,8,.) ... (2,14,.) (2,15,.) (2,16,6)
... (16 rows)
(16,1,B) (16,2,.) (16,3,.) (16,4,1) (16,5,7) (16,6,4) (16,7,F) ... (16,15,.) (16,16,E)
<<<SKETCHPAD END>>>

=== board 1 move v1 (3 op(s) remaining after this) ===
Move (board #1): Place 9 in cell r1c1 (row 1 from the top, column 1 from the left).
In your sketchpad, set that cell's tuple value to 9 and increment the VERSION line to 1. Leave every other cell unchanged.

<<<SKETCHPAD BEGIN>>>
VERSION: 1
(1,1,9) (1,2,8) (1,3,.) (1,4,G) (1,5,6) (1,6,7) (1,7,C) (1,8,.) ... (1,14,E) (1,15,.) (1,16,4)
... (rows 2-16 unchanged)
<<<SKETCHPAD END>>>
```

KV Store (a batch, the last query, and its answer)

```
=== set batch (keys 0-99) (25 op(s) remaining after this) ===
<<<SET-BATCH 0000#9226d9e1 BEGIN -- offload this whole block this turn>>>
SET K00000 = ember crimson saffron falcon marble quartz thistle mistral cobalt cinder ... #121536ed
... (100 SET lines)
<<<SET-BATCH 0000#9226d9e1 END>>>

=== get 23 (0 op(s) remaining after this) ===
GET K00114
(Report the value you stored for K00114 as an ANSWER block for that key.)

<<<ANSWER key=K00114>>>
garnet copper meadow mistral spruce pewter cinder pewter cobble saffron glacier ... #d17464a7
<<<ANSWER END>>>
```

Log Triage (a batch, the last query, and its answer)

```
=== log batch (lines 0-13) (35 op(s) remaining after this) ===
<<<LOG-BATCH 0000#21e46b32 BEGIN -- offload this whole block this turn>>>
2026-06-20T14:00:01Z [INFO] db-proxy req=49e7a0e2 timeout connection cache miss #f49c3100
... (14 log lines)
<<<LOG-BATCH 0000#21e46b32 END>>>

=== query 23 (0 op(s) remaining after this) ===
QUERY 23: How many [ERROR] log lines are from service "billing"?
(Answer with an ANSWER block for qid=23.)

<<<ANSWER qid=23>>>
4
<<<ANSWER END>>>
```

Figure 16 Operations as the agent receives them, from the smallest level of each task, shortened where marked. Blue text is the output the agent is expected to produce in response; it is graded from the agent's context. Needle Retention asks for no output: its needles are graded in the final context.

Method	Excerpt of the KV Store skill
CLM	*Mechanism:* Your context is mirrored to a file you can edit; changing that file changes what you are holding. *Each batch:* On the SAME turn you read a SET-BATCH, move the whole block out of your context and onto disk with this one command [a `python3` heredoc that moves the block to `/tmp/ctx_offload/` and leaves one placeholder line]. *Each GET:* Recover the value with `grep -h 'SET <key> =' /tmp/ctx_offload/*` and then print the ANSWER block. *Check:* No note, or a token count that did not drop, means the batch is still in your context.
ACM	*Mechanism:* `manage_context` takes everything since your previous `manage_context` call . . . and replaces it with one message [`summary_id: N`] `<summary>`. *Each batch:* Read the gauge after every batch; when it is above 22,000, call `manage_context` on your very next turn, before you release another batch. *Each GET:* Spend one turn on `query_memory(N, "the SET line for key K00084, verbatim")` with N the summary whose range covers the key. *Check:* What confirms a compression is the [`summary_id: N`] message and a [`context: ~N/M tokens`] readout lower than the turn before.
RLM	*Mechanism:* The REPL namespace persists: every variable and function you define survives from one operation to the next. That persistence is your store. *Each batch:* On the first operation define the store and the handler once; on every later operation only call them [a dictionary filled by a regular expression over the `SET` lines]. *Each GET:* A GET is answered by placing the ANSWER block in `answer["content"]`. *Check:* `print(out)` shows `stored N keys so far` growing by one batch per SET operation.
Context Folding	*Mechanism:* A branch begins from a copy of a context that already holds every batch and cannot delete anything from MAIN, so MAIN is as full after it as before. *Each batch:* On a SET-BATCH, run one command in MAIN and nothing else: `echo READY_FOR_NEXT_OP`. Do not open a branch. *Each GET:* Answer in MAIN . . . by finding the `SET <key> =` line in the batch in front of you and printing its value. *Check:* The [`context: ~N/M tokens`] readout should rise by the size of each batch and by almost nothing else.
Self-Compact	*Mechanism:* Compression happens only when C1=Y, C2=Y, C3=Y, N1=N. Then your whole history is replaced by . . . your summary. *Each batch:* On the SAME turn a SET-BATCH arrives, copy the whole block . . . to its own file. Answer [the probe] so that compression fires as soon as everything you have seen is on disk. *Each GET:* `grep -h "^SET K00084 = " /tmp/store/*.txt`, then print the value in an ANSWER block. *Check:* What confirms a store is the count `grep -c` prints back, equal to 100.
Summary	*Mechanism:* At three quarters of the budget (about 24.6K of 32,768 tokens) . . . everything except the system prompt and this task message is then replaced by one message. *Each batch:* On the SAME turn a SET-BATCH arrives, copy the whole block . . . to its own file. Write the summary as a pointer, not an inventory. *Each GET:* `grep -h "^SET K00084 = " /tmp/store/*.txt`, then print the value in an ANSWER block. *Check:* What confirms the store is the `grep -c` count coming back as 100.
Base	*Mechanism:* Every operation arrives as a message in this conversation and stays in your context for the rest of the run; there is no command, tool or edit that takes it back out. *Each batch:* On a SET-BATCH, run one command and nothing else. Do not copy batches to disk. *Each GET:* Find the `SET <key> =` line in the batch that is sitting in your context and print its value verbatim. *Check:* The [`context: ~N/M tokens`] readout should rise by the size of each batch and by almost nothing else.

Table 4 Excerpts of each method's KV Store skill. Every skill gives the same four kinds of guidance for its own mechanism: how the mechanism works, what to do with each batch, how to answer a query, and how to confirm that a step worked. Blue marks the concrete instruction for using the method's own tools; . . . marks omitted text and square brackets paraphrase code.

run with four seeds, or eight for selected levels to reduce variance. Results are in Figure 2.

TerminalBench 2.1. We use the 89 tasks of TerminalBench 2.1, each scored as pass or fail by its verifier. Qwen3.6-27B and Qwen3.5-9B are served with vLLM with thinking enabled and up to 4,096 generated tokens per call, with a 32,000-token budget; a request that would exceed it ends the run. Every method is limited to 64 turns, each shell command to 180 seconds and each task to four times its default time limit. Whenever a run ends, whether the agent finishes, reaches the turn or time limit, or exceeds the budget, the verifier scores the final state of the container. Results are in Figures 5 and 17.

TBLite. We use the OpenThoughts-TBLite tasks. The model is served with vLLM with up to 2,048 generated tokens per call and a 32,000-token budget. Turn and time limits and scoring are as for TerminalBench 2.1. Accuracy is the mean verifier reward. Results are in Figures 5 and 17.

BrowseComp-Plus. We use all 830 questions over the fixed BrowseComp-Plus corpus. The search tool returns the top 10 snippets of 512 tokens and the document reader is capped at 8,192 tokens. Models are served with vLLM with thinking enabled, temperature 0.7, top-p 0.95 and up to 4,096 generated tokens per call, with a 23,560-token budget and 100 turns; CLM's editing turns do not count toward the turn limit. When a request would exceed the budget, CLM rolls back the last turn and retries, up to six times. An answer is correct only if Qwen3.5-27B, judging at temperature 0 with the BrowseComp-Plus grading template, marks it correct and complete; unanswered questions count as wrong. Results are in Figures 5 and 17.

Math optimization problems. We use circle packing (26 circles in the unit square), Erdős' minimum overlap, the min–max distance ratio in the plane, and the Heilbronn triangle problem, with one run per method. The agent writes a program that an evaluator runs and scores. Every method uses Claude 4.6 Sonnet with a 32,000-token budget and stops after 100 evaluated attempts or five hours. OpenEvolve runs with a population of 60 in four islands; OpenEvolve-Agent uses a Mini-SWE-Agent proposer with 25 turns per candidate; CLM with subagents allows up to five subagents of 40 turns each. Results are in Table 1 and Figure 18.

EdgeBench-10. We use 10 of the 48 runnable public EdgeBench tasks (Table 5) with three seeds each. A run lasts 12 hours in a container without network access; each submission is graded by the EdgeBench judge on a 0–100 scale, and a run's score is the higher of its best graded submission and the grade of the final repository. Qwen3.6-27B is served with vLLM at temperature 0.7 with up to 8,192 generated tokens per call, and Claude 4.6 Sonnet runs through the API. The budget is 32,000 tokens; when a request would exceed it, the harness rolls back the last turn, up to 50 times, and no turn limit applies. CLM with subagents runs up to six concurrent subagents of 40 turns each. The 128K results in Figure 19 use a 128,000-token budget. Cost is prefix-reuse FLOPs per run, including the subagents' computation. Results are in Figures 6(a) and 19.

Task	Category	Language	What the verifier scores	Start: files / LOC
ad_placement_optimization	Combinatorial optimization	C++	solution score over judge cases	1 / 20
apple_incremental_game	Combinatorial optimization	Python	solution score over judge cases	3 / 101
graph_node_classification	Science & ML	Python	held-out accuracy (CPU-only judge)	2 / 418
grid_turing_robot	Combinatorial optimization	Python	solution score (lower is better)	3 / 440
juliet_vulnerability_analyzer	Software engineering	Python	hidden evaluator on the Juliet suite	1 / 9
openrct2_theme_park_ai	Games & simulators	JavaScript plugin	park value in a headless OpenRCT2 run	6 / 5,950
schemathesis_datagen_pipeline	Software engineering	Python	fraction of the test suite passing	416 / 112,663
triangulation_coloring_optimization	Combinatorial optimization	Python	coloring "ugliness" (lower is better)	5 / 431
vehicle_routing_time_windows	Combinatorial optimization	C/C++	CVRPTW solution quality	3 / 513
wesnoth_tactical_ai	Games & simulators	Python	headless Wesnoth matches	0 / 0

Table 5 EdgeBench-10 tasks. Scores are rescaled to 0–100 by the EdgeBench judge. Start: files and lines of code in the working directory before any edit.

Software World. Six agents, one per repository (`requests`, `urllib3` and four downstream packages), work in parallel for over 24 hours to make their repositories faster. The score is the geometric-mean speedup, in executed instructions, on 17 held-out CPU benchmarks from four downstream packages the agents never see; a benchmark that breaks counts as 1.0. Every agent runs in the Pi agent harness with Pi's default model and

context limit, GPT-5.6-Sol with a 272K-token context. Both swarms, with CLM and with summary compaction, use long wakes between work sessions. Cost is cumulative API spend in USD. Results are in Figure 6(b).

In-context steering. All three experiments use BrowseComp-Plus questions with Claude 4.6 Sonnet through the API, the unmodified CLM harness, no budget reminders, and a run that ends when a request exceeds the budget. Each behavior is induced by an instruction appended after the task message and compared with a no-instruction control on the same questions; confidence intervals are BCa bootstrap intervals over paired differences. Threshold: a 48,000-token budget and 200 turns on 30 long questions (at or above the 60th percentile of turn count without instruction), with the instruction to compact once the context passes 16K, 24K or 32K tokens; we report the median context size at the first compaction. Boundaries: sessions of four chained questions with a 24,000-token budget and 1,000 turns, 189 sessions per condition, 131 of them with a boundary inside the session; we report the rate of compaction within two turns of a question boundary. Backup: a 16,000-token budget on 200 questions, 91 of them paired for analysis; we report the fraction of edits preceded by a full copy of the context. Results are in Figure 7. The appended instructions are, for the threshold (with $Y \in \{16{,}000, 24{,}000, 32{,}000\}$):

> Monitor your context size: every tool result reports your current size. Whenever it exceeds Y tokens, immediately compact your context down to about 4000 tokens. For example: rewrite /tmp/.live_ctx/LIVE_CTX_MAIN.txt to keep only your key findings and current plan (about 4000 tokens).

for boundaries:

> Please only compress at the boundaries of subtasks. Once a subtask is accomplished, throw out all the context for that subtask. For example: after you write /workspace/answer_1.txt, rewrite /tmp/.live_ctx/LIVE_CTX_MAIN.txt to drop the Question 1 material, keeping only your answer_1 conclusion.

and for backup:

> Before every edit to your context, you must back up the current context file into a new file inside a new folder named compaction_backup; never overwrite or delete earlier backups.

In-context evolution. The loop runs on ContextBench. Each proposed skill is first run on six training instances; if its accuracy there is at least that of the current frontier, it is run on twelve more, and then on a development split of 102 instances per task with one seed, the only split used to decide frontier membership and selection. For assisted evolution, a test split of 102 instances with three seeds is evaluated once after the archive is frozen. The agent is Qwen3.6-27B served with vLLM, with a 32,000-token budget, a 4,096-token reserve and 240 turns; a run that exceeds the budget ends. In assisted evolution, Claude Fable 5.1 proposes the skills; in self-evolution, Claude Opus 5 is both the agent and the proposer. Four proposers work in parallel; each reads at least ten rollouts of the current skill, including five successful and failed runs on the same instance, and writes at least four full rewrites, each with a predicted effect on accuracy and cost. A lineage stops after five consecutive proposals without improvement on the development split. Rewards come from the deterministic task graders. A skill enters the Pareto frontier if no earlier skill, the starting point included, is at least as accurate and at least as cheap; the selected skill is the most accurate on the development split, with ties within one standard error broken by cost. Cost is prefix-reuse FLOPs per task over runs that finish within the budget for Qwen3.6-27B, and USD per task for Opus 5. Results are in Figures 8 and 20.

Reinforcement learning. We train Qwen3.5-9B on 3,040 OpenResearcher deep-research prompts using GRPO. Training uses truncated importance sampling, a low-variance KL loss with coefficient 0.01, and dynamic sampling that removes groups with no reward variation. Each step samples 8 prompts with 32 rollouts per prompt. We use a learning rate of 10^{-6}, weight decay of 0.1, and a clipping range of $[0.2, 0.28]$, and train for 70 steps. Rollouts are served with SGLang at temperature 0.7 and top-p 0.95, with thinking enabled and up to

Assisted evolution: Qwen3.6-27B = agent,
Claude Fable 5.1 = proposer.
In self-evolution, Claude Opus 5 = both roles.
Budget is per individual agent run, not total experiment

4,096 generated tokens per turn. CLM uses a 28K context budget with a 2,048-token reserve and up to 80 turns, excluding editing turns from the count; the summary harness compacts at 28,672 tokens and runs for up to 100 turns. We use a binary task reward from GPT-5.4-nano with the DeepSearchQA rubric. For CLM, we additionally apply the efficiency advantage from Section 4.2 with weight 0.25 to context-management tokens, together with penalties for failed tool calls and malformed outputs. The summary harness is trained with the task reward alone. Training uses 16 H200 GPUs for the policy and 48 for rollout generation. We select the checkpoint with the highest accuracy on 500 held-out OpenResearcher questions, breaking ties within one standard error in favor of the earlier checkpoint, and evaluate it on all 830 BrowseComp-Plus questions using the same judge. Prefix-reuse FLOPs are computed with Qwen3.5-9B model constants and exact token-level prefix matching against the previous turn. Results are reported in Table 2 and Figure 21.

F Supplementary Results

TerminalBench 2.1, TBLite and BrowseComp-Plus with models of different sizes. Figure 17 places Qwen3.5-9B next to Qwen3.6-27B on the three benchmarks of Figure 5. CLM leaves the decision of when and how to edit the context to the model, so its gains grow with the model's ability to make that decision. With Qwen3.6-27B, CLM lies on the Pareto frontier of all three benchmarks and reaches the highest accuracy on BrowseComp-Plus (59.4%). With Qwen3.5-9B, CLM reaches 39.9% on BrowseComp-Plus, above the summary harness (37.7%), and the smaller model edits its context less often: on TerminalBench 2.1, Qwen3.5-9B edits its context 1.4 times per task on average and makes no edit in half of the tasks, while Qwen3.6-27B edits 2.6 times per task. The median peak context is correspondingly higher for Qwen3.5-9B, 30.2K tokens of the 32K limit compared with 17.6K for Qwen3.6-27B.

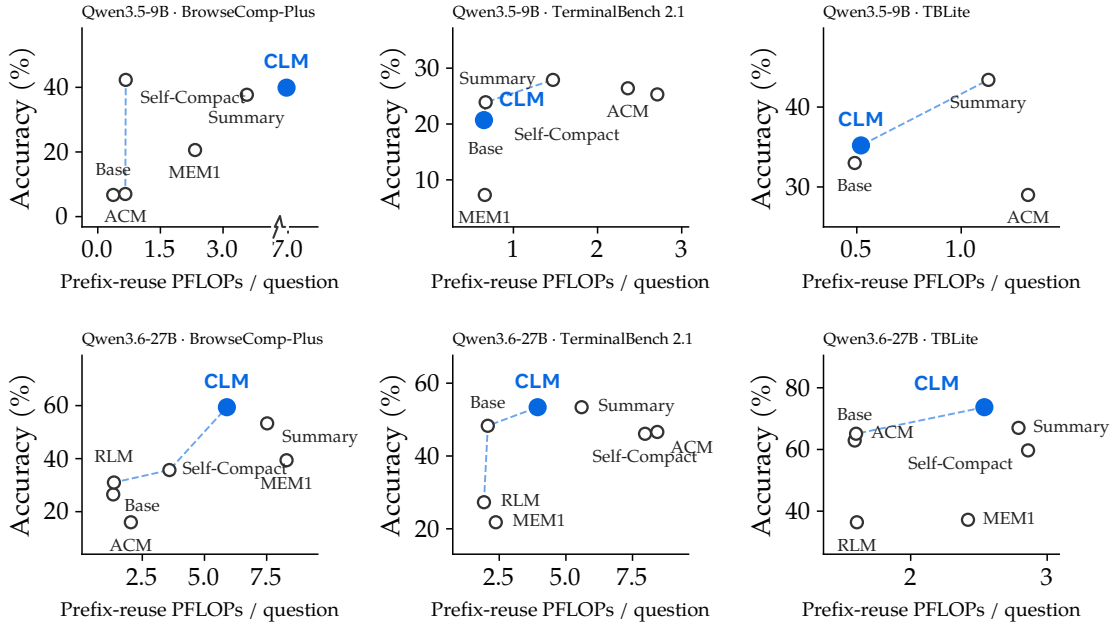
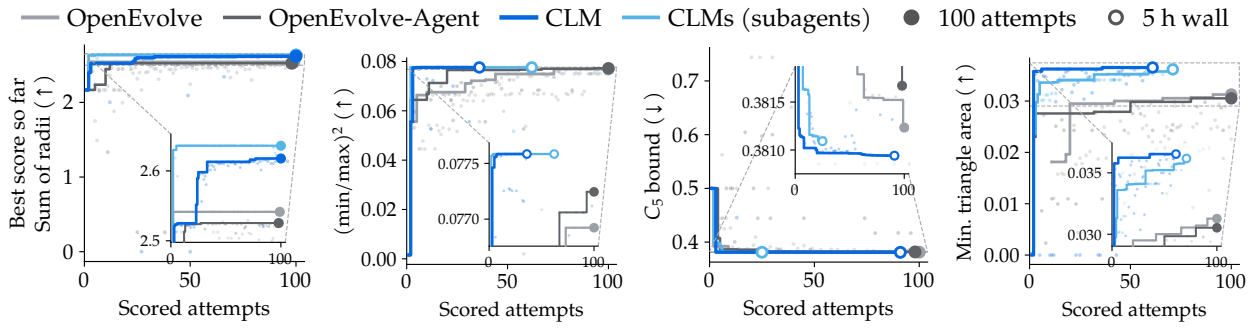


Figure 17 Qwen3.5-9B and Qwen3.6-27B. Accuracy against compute for Qwen3.5-9B (top) and Qwen3.6-27B (bottom) on BrowseComp-Plus, TerminalBench 2.1 and TBLite with a 32K context limit. Cost is prefix-reuse PFLOPs per question; dashed lines indicate the Pareto frontier.

Math optimization problems. Figure 18 shows the best score so far against the number of scored attempts for each method on the four problems.

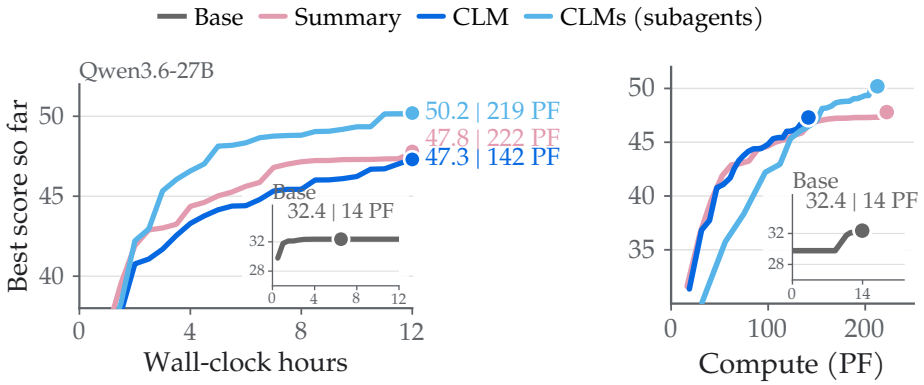
EdgeBench-10 with a 128K context budget. Figure 19 repeats the EdgeBench-10 comparison of Figure 6 with a 128K context budget. The three methods that manage context keep improving over the twelve hours, while the base harness stops improving within the first two hours. At a 32K budget, CLM with and without



(a) Circle packing (↑). **(b)** Min-max/min-dist (↑). **(c)** Erdős min-overlap (↓). **(d)** Heilbronn triangle (↑).

Figure 18 Best-so-far score versus evaluator-scored attempts on four open optimization problems. All runs use Claude 4.6 Sonnet with a 32K context limit and stop after 100 attempts or five hours. Lines show the best score so far, dots individual scored candidates, and insets the final-score range.

subagents end within 0.4 points of each other (44.2 and 44.6). At 128K, CLM with subagents reaches 50.2, compared with 47.3 for CLM and 47.8 for summarization, using 219, 142 and 222 prefix-reuse PFLOPs per trial.



(a) Score against time. **(b)** Score against compute.

Figure 19 EdgeBench-10 single-repository optimization with a 128K context budget. Qwen3.6-27B, ten tasks, three seeds. End labels give final scores and mean compute per trial (PF = prefix-reuse PFLOPs); insets show the base harness.

In-context evolution. Figure 20 extends Figure 8 to all four tasks of ContextBench. In assisted evolution, starting without any context-management instruction, the selected skill raises development accuracy from 97.6% to 100.0% on Needle Retention, from 45.3% to 65.8% on Sudoku Sketchpad, from 22.3% to 83.8% on KV Store and from 0.0% to 100.0% on Log Triage. On the held-out test split, the selected KV Store skill raises accuracy from 38.3% to 74.2%. In self-evolution, Opus 5 starts between 94% and 100% accuracy, and the evolved skills either reduce cost at the same or higher accuracy or raise accuracy further.

Reinforcement learning. Figure 21 shows accuracy and compute on BrowseComp-Plus during training for CLM and the summary harness, each trained with and without the FLOPs reward.

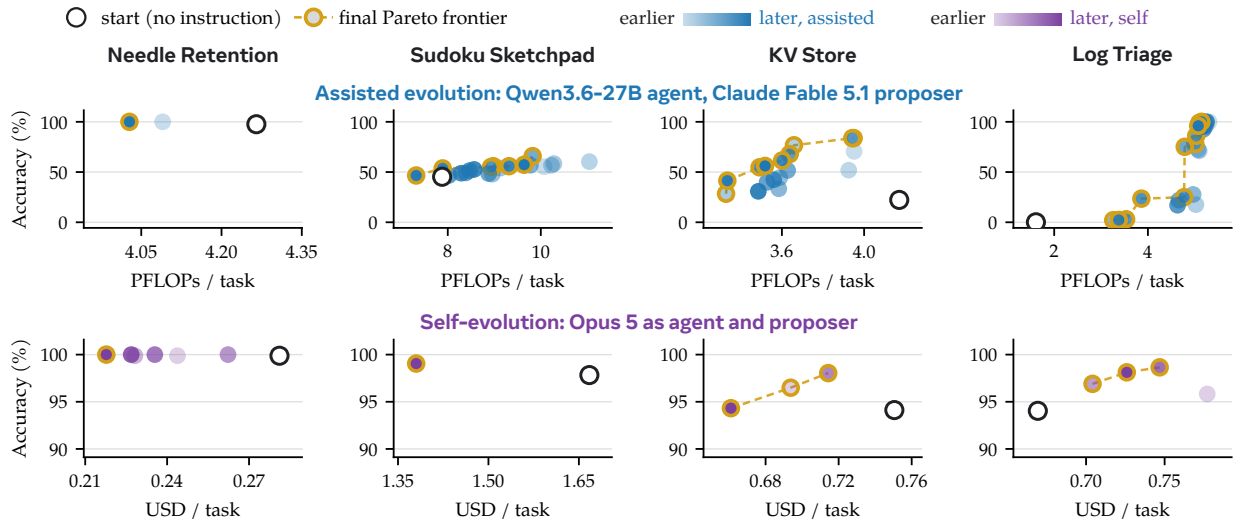


Figure 20 Evolving context-management skills on ContextBench (32K budget), all four tasks. **Assisted evolution** (top): Qwen3.6-27B serves as the agent, while Claude Fable 5.1 proposes the skills; the x-axis shows prefix-reuse PFLOPs per task over solved runs. **Self-evolution** (bottom): Opus 5 serves as both the agent and the skill proposer; the x-axis shows gateway cost per task in USD. Skills are proposed based on training-split rollouts and never use the held-out evaluation set.

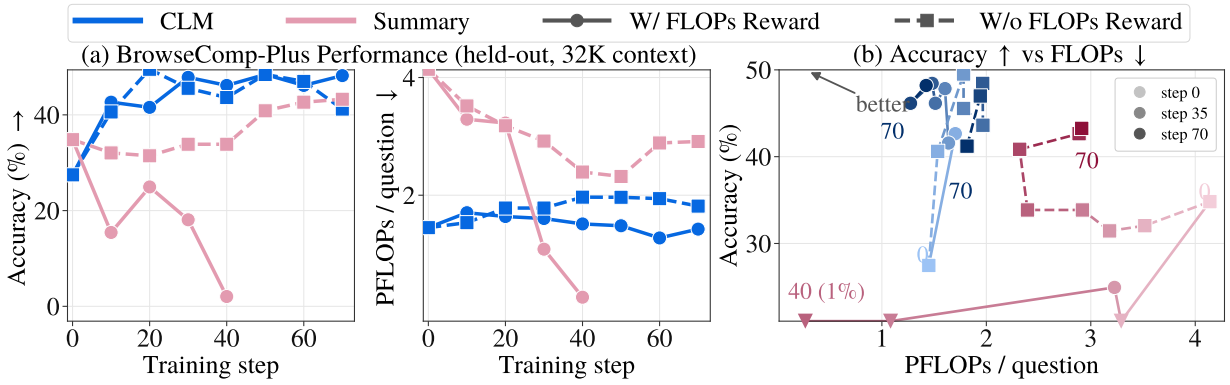


Figure 21 RL training curves. Qwen3.5-9B trained on OpenResearcher and evaluated on BrowseComp-Plus with a 32K context limit, through training step 70. (a) Accuracy and prefix-reuse PFLOPs per question against training step. (b) Accuracy against compute, shaded from light to dark by training step.

G Analysis on the Context Length Awareness of Existing LMs

An important capability for CLMs is context length awareness: the ability to estimate how much of the context budget has been consumed and to decide when context editing or offloading is needed. Without such awareness, agents may rely on frequent external system interventions, which can leave stale or redundant information in future turns after context editing.

We probe context length awareness with a simple diagnostic. We provide each model with prompts of varying lengths and ask it to estimate the number of tokens in the current context. Figure 22 compares the models' estimated context lengths against the actual prompt lengths. Interestingly, models tend to predict recurring bucketed values in the longer-context regime, such as 6.2K, 9.8K, or 10.4K tokens. We conjecture that these bucketed estimates may reflect token-counting patterns seen during pretraining or post-training. We further observe that Claude-4.6-Sonnet tends to underestimate context length, while GPT-5.4 achieves the strongest alignment between estimated and actual token counts among the three models. Furthermore, token-count hints improve context-length estimation, and hints closer to the estimation point are more effective. These results indicate that existing LMs have limited context-length awareness at long context lengths, where environmental hints can help substantially. Future CLM training may incorporate such signals to improve context-length awareness.

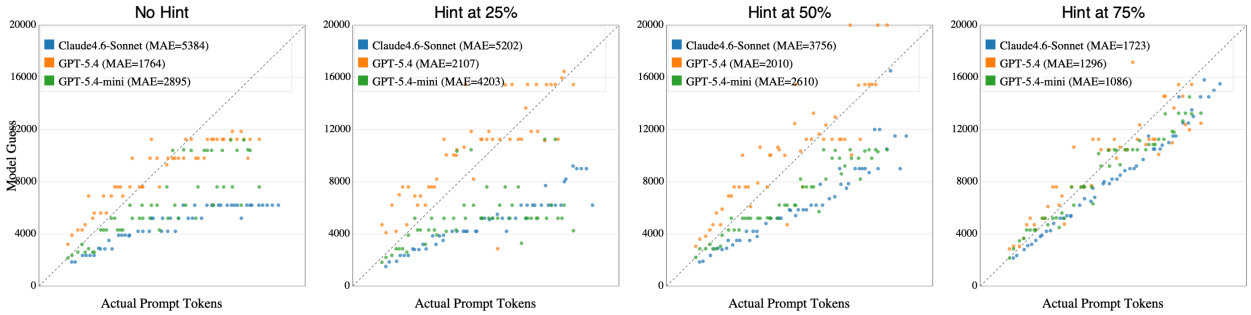


Figure 22 Context length awareness. Each point compares a model's estimated context length with the provider-reported prompt length for the same 50 inputs. Panels vary the available hint: none or a token-count anchor at 25%, 50%, or 75% of the input. The dashed line indicates perfect calibration; legend values report mean absolute error (MAE) in tokens.